

目 录

第一部分 实验要求	3
第二部分 需求分析	3
2.1 任务范围	3
2.2 输入输出	4
2.3 系统功能	5
2.4 测试数据	7
第三部分 概要设计	10
3.1 任务分解	10
3.2 数据类型定义	13
第四部分 详细设计	20
4.1 词法分析器详细设计	22
4.2 语法分析器详细设计	26
4.3 语义分析和中间代码生成器详细设计	32
第五部分 算法分析框图	40
5.1 总程序算法流程图	40
5.2 词法分析算法流程图	41
5.3 语法分析算法流程图	42
5.4 语义分析及中间代码生成算法流程图	43
第六部分 调试分析与错误处理	45
6.1 调试测试	45
6.2 问题发现与解决	46
第七部分 静态语义错误的诊断和处理	43
7.1 变量重定义	43
7.2 数组使用越界	44
7.3 声明前使用变量	46
第八部分 静态语义错误的诊断和处理	47
8.1 类型检查	47
8.2 指针和引用类型的检查	49
8.3 数组越界检查	49
8.4 内存泄漏检查	50
8.5 高级语言的中间代码	50
第九部分 总结与收获	52
9.1 类型检查	52
9.2 指针和引用类型的检查	52
参考文献	53

一 实验要求

在 LR(1) 分析方法得到的语法分析器基础上，编写一个类 C 语言的语义分析程序：

- (1) 给出源程序的语义分析结果，实现中间代码生成（建议以四元式作为中间代码）；
- (2) 注意静态语义错误的诊断和处理；
- (3) 在此基础上，考虑更为通行的高级语言的语义检查和中间代码生成所需要注意的内容，并给出解决方案。

二 需求分析

2.1 任务范围

本项目将在实现词法和语法分析器的基础上对语义分析和中间代码生成器的实现展开研究。我们采用语法制导的方式，在语法分析的同时生成中间代码（以四元式的形式表示）。本实验实现的部件单元为编译器前端部分的最后一个模块，其在编译流程中的位置如下图所示。

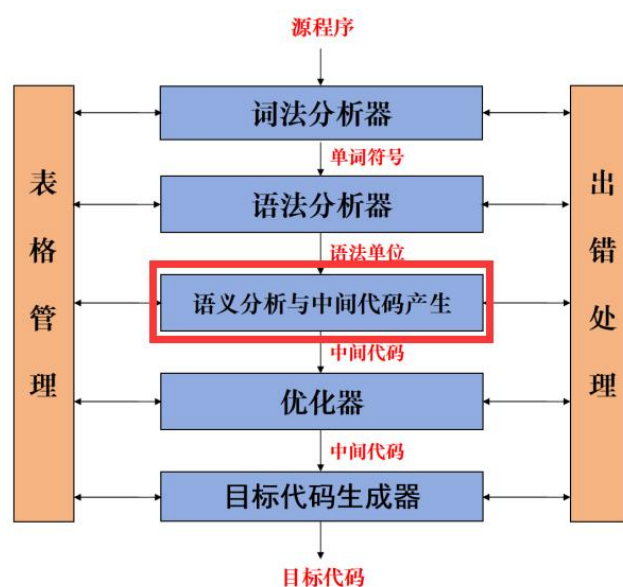


图 1 编译器的结构示意图

2.1.1 语义分析的主要工作

语义分析可以在比语法分析更高级的层次上对代码中的静态元素进行类型、名称、作用域的分析。

在语法分析的基础上进行静态语义检查。其关注程序的静态性质，如变量的声明和使用、类型是否匹配等。静态语义检查可以分为类型检查、控制流检查、一致性检查、相关名字检查和名字的作用域分析等。其中控制流检查需要分析程序的控制流结构，确保在不同情况下

程序的执行流程是合法的。名字的作用域分许可以 、确定程序中各个标识符的有效范围，即在哪些地方可以访问这些标识符。

语义分析模块在编译流程中的作用如图所示：

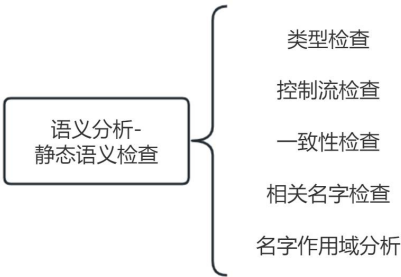


图 2 语义分析作用

2.1.2 中间代码生成的主要工作

中间代码是源代码的更高级抽象表示，其脱离了与具体硬件平台的联系，够帮助编译器在编译的下一阶段进行更通用的处理。

经过静态语义分析检查后，根据语法分析生成的抽象语法树（AST）和符号表，以及类型信息和控制流信息生成中间代码。中间代码有多种表达形式，包括三地址码、四元式、中缀表达式、后缀表达式等。此处实现我们采用四元式进行中间代码的表示。在递归遍历 AST 的过程中进行类型检查并且生成四元式。对源码中的控制流结构同时还要引入标签或者临时变量。

语义分析和中间代码生成在编译流程中的位置如下图所示

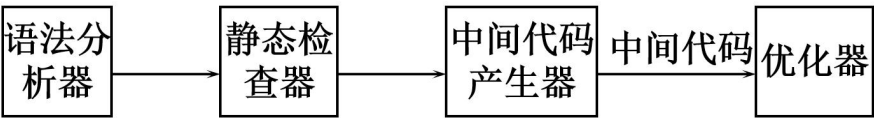


图 3 语义分析（静态语义检查）与中间代码产生器在编译中的流程

2.2 输入输出

2.2.1 系统输入

本试验在词法和语法分析的基础上进行，程序提供两种输入手段，输入为用户上传 .cpp / .cc / .txt 或其他文本文件，文件中为类 C 语句；或者为用户在程序文本框中输入的类 C 代码段。系统将根据文本框中的最终内容进行词法分析、语法分析、中间代码生成等操作。

2.2.2 系统输出

系统输出相较于词法分析和语法分析试验中提供的词法分析结果、语法分析 ACTION / GOTO 表、移进规约过程、语法树外，添加了 First 集输出、四元式分析结果、符号栈动作和变化过程展示。其中若生成失败则返回对应信息，回显到对应文本框中。同时程序在控制台将输出终端版 Item 集合、语法树绘制、操作是否成功信息回显等。

2.3 系统功能

本系统的目标是实现类 C 语法的语义分析和中间代码生成器，并创建用户界面来实现该功能与用户之间的交互。系统后台会预定义一个类 C 语法，系统可对输入的词法单元串进行语法分析。若分析成功，将展示相应的语法分析树、详细的词法和语法分析过程、生成的中间代码；否则将提供相应的错误提示（如果是因上传文件不是 LR(1)型语法，系统可以进行甄别）。

系统注重用户界面的设计，以提供友好的交互体验。用户可以直接上传文件或输入类 C 语法的代码，点击“分析代码”便在界面上查看各类分析结果，具体按钮和截图功能展示如下。

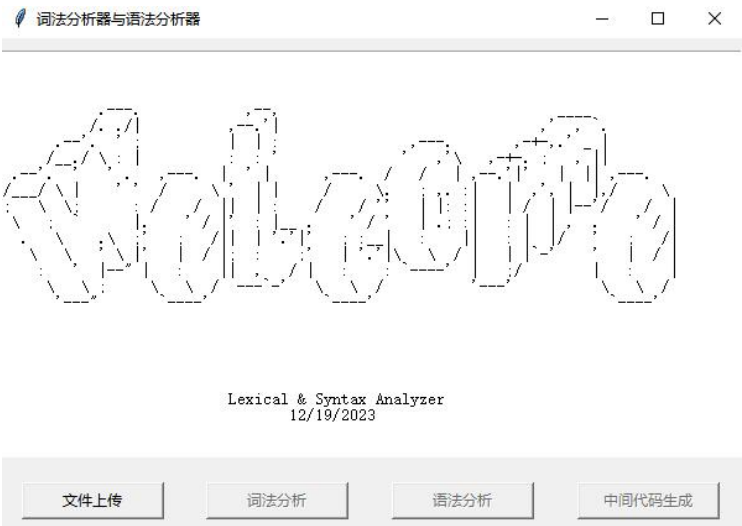


图 4 系统初始界面，未上传文件时，词法分析和语法分析按钮无法点击

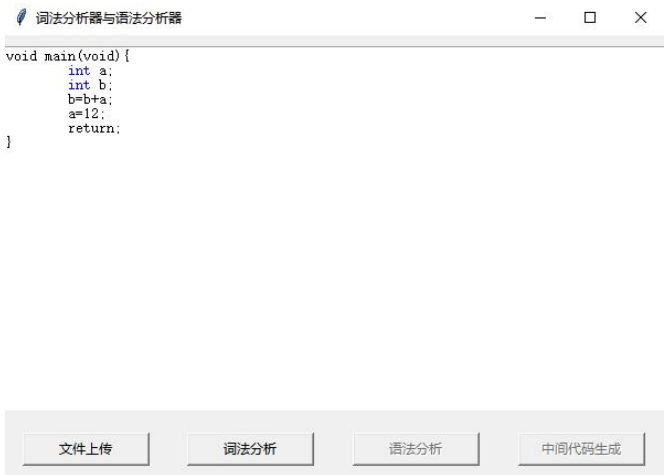
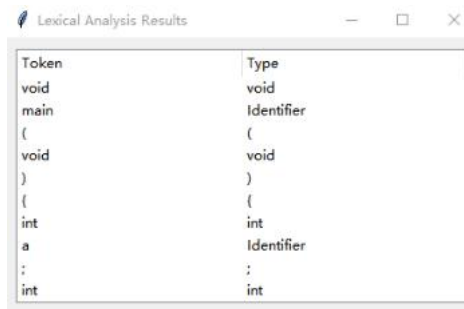
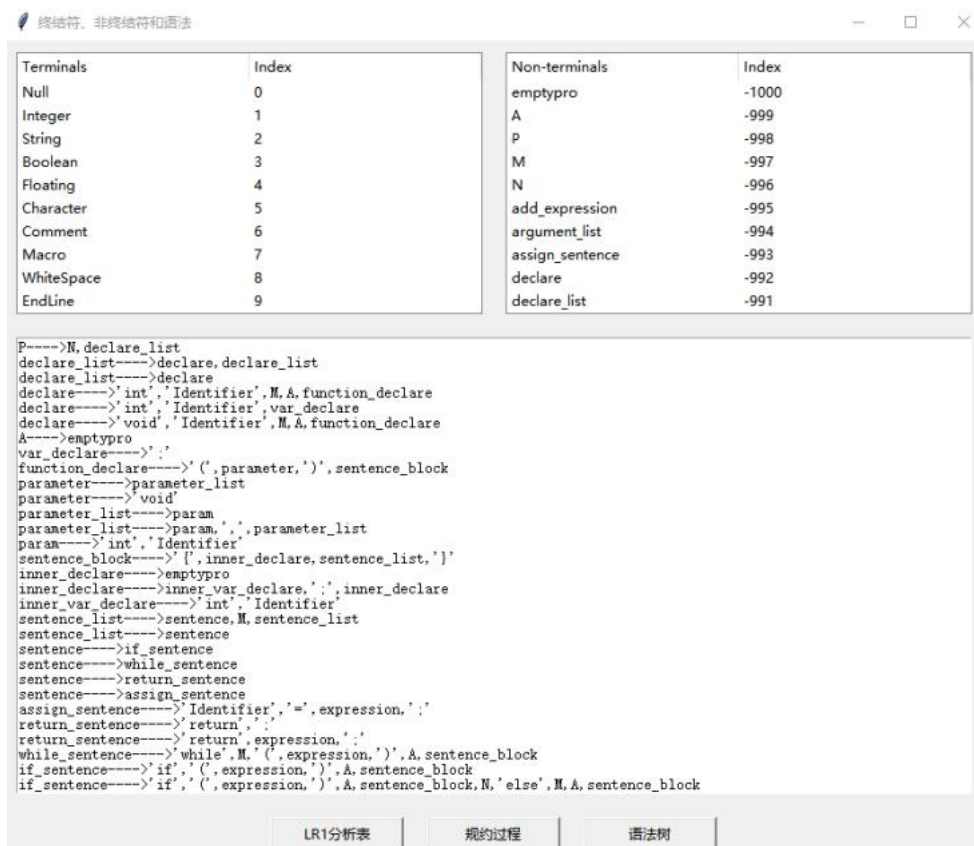


图5 上传代码后，关键字高亮，词法分析按钮使能



Token	Type
void	void
main	Identifier
{	{
void	void
}	}
{	{
int	int
a	Identifier
;	;
int	int

图6 点击词法分析按钮后弹窗展示词法分析结果，同时主界面语法分析按钮使能



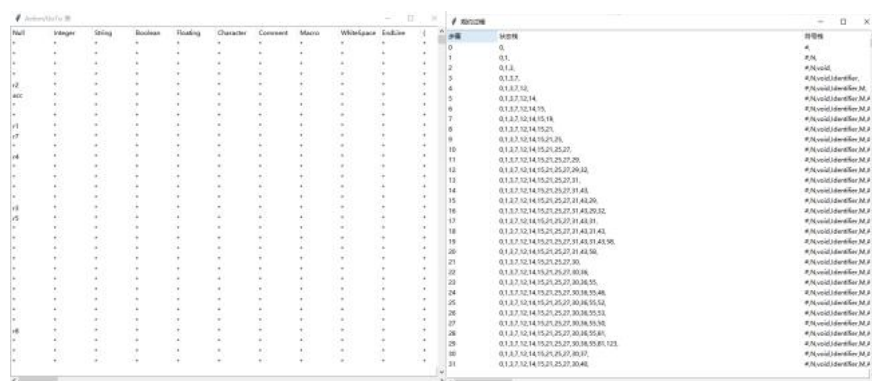
Terminals	Index
Null	0
Integer	1
String	2
Boolean	3
Floating	4
Character	5
Comment	6
Macro	7
WhiteSpace	8
EndLine	9

Non-terminals	Index
emptypro	-1000
A	-999
P	-998
M	-997
N	-996
add_expression	-995
argument_list	-994
assign_sentence	-993
declare	-992
declare_list	-991

```
P---->M, declare_list
declare_list---->declare, declare_list
declare_list---->declare
declare---->'int', 'Identifier', M, A, function_declare
declare---->'int', 'Identifier', var_declare
declare---->'void', 'Identifier', M, A, function_declare
A---->emptypro
var_declare---->' '
function_declare---->'(', parameter, ')', sentence_block
parameter---->parameter_list
parameter---->'void'
parameter_list---->param
parameter_list---->param, ',', parameter_list
param---->'int', 'Identifier'
sentence_block---->'{' , inner_declare, sentence_list, '}'
inner_declare---->emptypro
inner_declare---->inner_var_declare, ' ', inner_declare
inner_var_declare---->'int', 'Identifier'
sentence_list---->sentence, M, sentence_list
sentence_list---->sentence
sentence---->if_sentence
sentence---->while_sentence
sentence---->return_sentence
sentence---->assign_sentence
assign_sentence---->'Identifier', '=', expression, ';'
return_sentence---->'return', ' '
return_sentence---->'return', expression, ' '
while_sentence---->'while', M, '(', expression, ')', A, sentence_block
if_sentence---->'if', '(', expression, ')', A, sentence_block
if_sentence---->'if', '(', expression, ')', A, sentence_block, N, 'else', M, A, sentence_block
```

LR1分析表 规约过程 语法树

图7 语法分析窗口，左上方为终结符，右上方为非终结符，下方为语法分析结果



Null	Integer	String	Boolean	Floating	Character	Comment	Macro	WhiteSpace	EndLine
+	+	+	+	+	+	+	+	+	+
-	-	-	-	-	-	-	-	-	-
*	*	*	*	*	*	*	*	*	*
/	/	/	/	/	/	/	/	/	/
%	%	%	%	%	%	%	%	%	%
~	~	~	~	~	~	~	~	~	~
^	^	^	^	^	^	^	^	^	^
&	&	&	&	&	&	&	&	&	&
@	@	@	@	@	@	@	@	@	@
#	#	#	#	#	#	#	#	#	#
\$	\$	\$	\$	\$	\$	\$	\$	\$	\$
%	%	%	%	%	%	%	%	%	%
~	~	~	~	~	~	~	~	~	~
^	^	^	^	^	^	^	^	^	^
&	&	&	&	&	&	&	&	&	&
@	@	@	@	@	@	@	@	@	@
#	#	#	#	#	#	#	#	#	#
\$	\$	\$	\$	\$	\$	\$	\$	\$	\$

非终结符	终结符	非终结符
0	0	A
1	1	A
2	2	A
3	3	A
4	4	A
5	5	A
6	6	A
7	7	A
8	8	A
9	9	A
10	10	A
11	11	A
12	12	A
13	13	A
14	14	A
15	15	A
16	16	A
17	17	A
18	18	A
19	19	A
20	20	A
21	21	A
22	22	A
23	23	A
24	24	A
25	25	A
26	26	A
27	27	A
28	28	A
29	29	A
30	30	A
31	31	A

图8 LR(1)分析表窗口与规约过程窗口

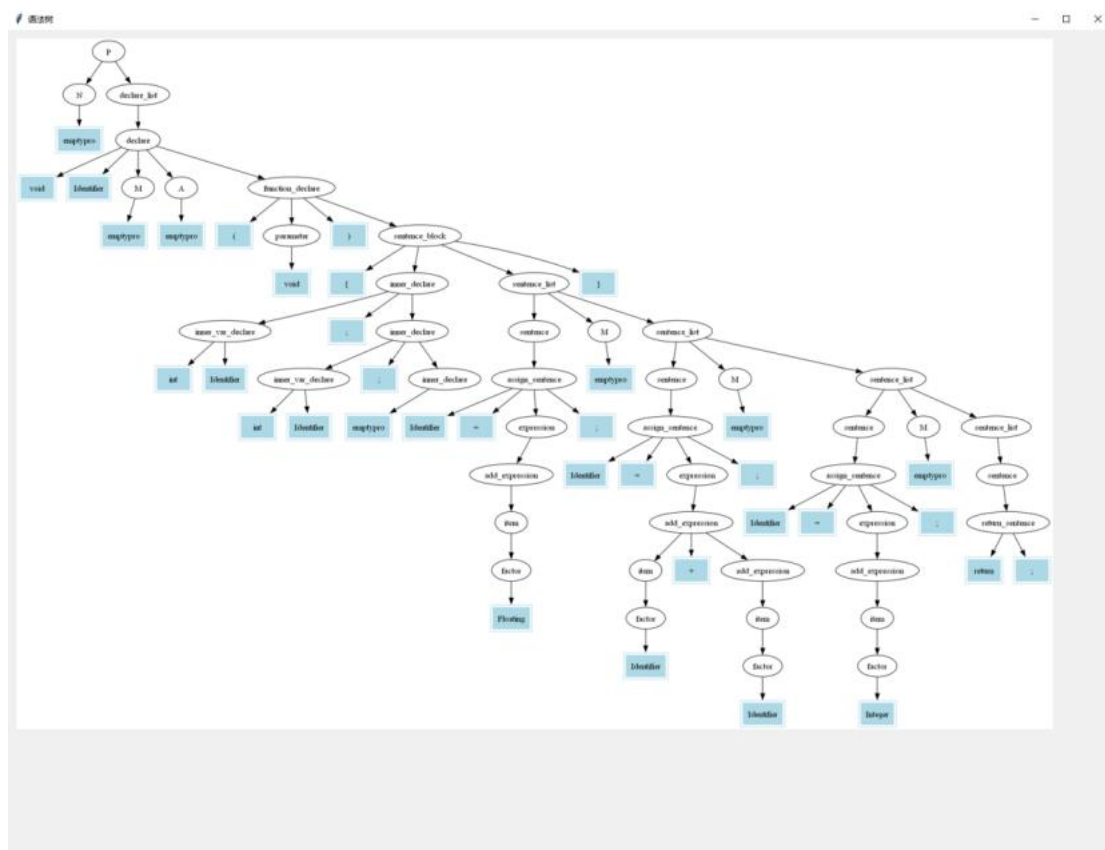


图 9 语法树分析窗口

Source Code	Quaternions	Sem Stack
<pre>int main() { int b; int a; a = 0; b = 2.0+4*7; while (a > 3) { a = a + 1; b = b + a; } if (a <= b) { a = 1+2+3-6/3+2*(3+4); b = a + 2; } else { b = 9; a = a*b + b+(0-8*a); } }</pre>	<pre>Accept! Quaternions information: (=, 0, ., a) (*, 4, 7, 0_TEMP) (+, 2, 0, 0_TEMP, 1_TEMP) (=, 1_TEMP, ., b) (0_LABEL, ., .) (>, a, 3, 2_TEMP) (J+, 2_TEMP, 0, 1_LABEL) (J, ., 2_LABEL) (1_LABEL, ., .) (+, a, 1, 3_TEMP) (=, 3_TEMP, ., a) (+, b, a, 4_TEMP) (=, 4_TEMP, ., b) (J, ., 0_LABEL) (2_LABEL, ., .) (<=, a, b, 5_TEMP) (J+, 5_TEMP, 0, 3_LABEL) (J, ., 4_LABEL) (3_LABEL, ., .) (*, 2, 3, 6_TEMP) (+, 1, 6_TEMP, 7_TEMP) (/, 6, 3, 8_TEMP) (-, 7_TEMP, 8_TEMP, 9_TEMP) (+, 3, 4, 10_TEMP) (*, 2, 10_TEMP, 11_TEMP) (+, 9_TEMP, 11_TEMP, 12_TEMP) (=, 12_TEMP, ., a) (+, a, 2, 13_TEMP) (=, 13_TEMP, ., b) (J, ., 5_LABEL) (4_LABEL, ., .) (=, 9, ., b) (*, a, b, 14_TEMP) (+, 14_TEMP, b, 15_TEMP) (*, 8, a, 16_TEMP) (-, 0, 16_TEMP, 17_TEMP) (+, 15_TEMP, 17_TEMP, 18_TEMP) (=, 18_TEMP, ., a) (6_LABEL, ., .)</pre>	<pre>Symbol stack information: Inserted element: int KEYWORD Stack Information: 8 int 0 \$ Inserted element: main ID Stack Information: 11 ID 5 type_specifier 0 \$ Inserted element: (DELIMITER Stack Information: 12 (11 ID 5 type_specifier 0 \$ Inserted element:) DELIMITER Stack Information: 14) 12 (11 ID 5 type_specifier 0 \$ Inserted element: (DELIMITER Stack Information: 16 (14) 12 (11 ID 5 type_specifier 0 \$</pre>

图 10 语义分析过程与四元式表示

2.4 测试数据

我们预先构建了 7 个测试文件，分三个单元进行测试：

① 其中正确数据 1.c、2.cpp 为正确数据，涵盖了从简单定义到浮点数到函数嵌套等逐级递增的复杂度；

- ② 错误数据中 3.cpp 中存在未声明的变量
- ③ 4.c 中有调用函数时形参和实参不匹配的错误
- ④ 5.cpp 中有类型错误（将整数赋值给 void 变量）
- ⑤ 6.txt 中有变量重定义错误。

测试文件内容展示如下：

1. c

```
1. int a;  
2. int p(void){  
3.     int b;  
4.     return 2;  
5. }
```

2. cpp

```
1. int a;  
2. int b;  
3. int program(int a, int b, int c){  
4.     int i;  
5.     int j;  
6.     i = 0;  
7.     if (a > (b + c)){  
8.         j = a + (b * c + 1);  
9.     }  
10.    else{  
11.        j = a;  
12.    }  
13.    a = a + 10;  
14.    while (i <= 100){  
15.        i = j * 2;  
16.        j = i;  
17.    }  
18.    return i;  
19.}  
20.  
21.int demo(int a){  
22.    a = a + 2;  
23.    return a * 2;  
24.}  
25.  
26.void main(void){  
27.    int a;
```

```
28. int b;  
29. int c;  
30. a = 3;  
31. b = 4.23;  
32. c = 2+3e-2;  
33. c = b + a;  
34. a = program(a, b, demo(c));  
35. return;  
36. }
```

3. cpp

```
1. int main(void){  
2.     a=1;  
3.     return -1;  
4. }
```

4. c

```
1. int fun(){  
2.     return 10;  
3. }  
4.  
5. int main(void){  
6.     int a=fun(20);  
7.     return -1;  
8. }
```

5. cpp

```
1. int main(){  
2.     int a=1;  
3.     void b=a;  
4.     return -1;  
5. }
```

6. txt

```
1. int main(){  
2.     int a=1;  
3.     int a=2;  
4.     return -1;  
5. }
```


测试文件将测试系统在不同的代码复杂度与错误情况下的表现，如包含了不同文件格式、存在额外空格回车制表符、同时存在整数浮点数等等。为了测试错误情况，我们在构建测试数据时主要关注错误的词法单元序列，以此检验代码和系统的正确性。相比于词法和语法分析器测试文件，我们对应构造了对应静态语义分析错误测试文件，以测试系统在语义分析及中间代码生成任务上的表现。

三 概要设计

3.1 任务分解

3.1.1 词法分析与语法分析

该部分与第一次实验叙述重合，此处仅选取关键点进行说明。

3.1.1.1 词法分析

1. 读入类 C 文件。将文件中内容的以字符串的形式读入变量中；
2. 分析字符、字符串、分隔符、关键字、运算符等词法单元。我们以类似哈希表的数据结构预先定义了终结符 VT 与非终结符 NT 集，便于词法分析过程识别和生成相应的终结符分析。
3. 分析整数、浮点数、标识符和其他单词。本项目中构建了 3 个 DFA 来分别识别整数、浮点数、标识符；我们采用字典树，利用其前缀匹配功能进行关键字的快速查找。同时在实现过程中针对字典树时间空间表现进行了权衡和优化。

4. 输出词法分析结果。

本实验中详细类 C 语法规则如下：

1. 关键字: `int | void | if | else | while | return`
2. 标识符: `字母 (字母 | 数字)*` (注: 不与关键字相同)
3. 数值: `数字 (数字)*`
4. 赋值号: `=`
5. 算符: `+ | - | * | / | = | == | > | >= | < | <= | !=`
6. 界符: `;`
7. 分隔符: `,`
8. 注释号: `/* */ | //`
9. 左括号: `(`
10. 右括号: `)`
11. 左中括号: `[`
12. 右中括号: `]`
13. 左大括号: `{`
14. 右大括号: `}`
15. 字母: `| a | ... | z | A | ... | Z |`
16. 数字: `0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |`
17. 结束符: `#`

编译器持关键字: `int, double, char, float, break, continue, do, while, if, else, for,`

void , return

3.1.1.2 语法分析

语法分析器模块沿用实验一对应模块,并根据静态语义检查和中间代码生成要求在规约过程中添加翻译模块。基础语法分析器读取预先定义的语法文件,其中包含了文法的产生式、终结符、非终结符等规则。通过计算 First 集和 Follow 集,为后续的 LR(1)分析器构建提供必要的信息。利用深度优先搜索算法,求解闭包和 LR(1)项目集族,构建状态转移图表示不同项目集之间的转移关系。同时生成 LR(1)集族的状态转移信息,包括 Action 表和 Goto 表,用于处理终结符和非终结符的移进、规约和转移操作。

语法分析过程中,记录总规约过程,包括操作栈、数字栈和行动说明,将结果写入文件以供查看。最后通过规约过程构建语法树,为后续的语义分析和代码生成阶段提供分析条件。本实验中使用的语法规则如下:

1. Program ::= <声明串>
2. <声明串> ::= <声明> { <声明> }
3. <声明> ::= **int** <ID> <声明类型> | **void** <ID> <函数声明>
4. <声明类型> ::= <变量声明> | <函数声明> | <数组声明>
5. <变量声明> ::= ;
6. <函数声明> ::= '(' <形参> ')' <语句块>
7. <数组声明> ::= '[' (数字)+ ']' { '[' (数字)+ ']' }
8. <形参> ::= <参数列表> | **void**
9. <参数列表> ::= <参数> { , <参数> }
10. <参数> ::= **int** <ID>
11. <语句块> ::= '{' <内部声明> <语句串> '}'
12. <内部声明> ::= 空 | <内部变量声明>; { <内部变量声明>; }
13. <内部变量声明> ::= **int** <ID>
14. <语句串> ::= <语句> { <语句> }
15. <语句> ::= **if** 语句 | **while** 语句 | **return** 语句 | <赋值语句>
16. <赋值语句> ::= <ID> '=' <表达式>; | <数组> '=' <表达式>;
17. <return 语句> ::= **return** [<表达式>] ; (注: [] 中的项表示可选)
18. <while 语句> ::= **while** '(' <表达式> ')' <语句块>
19. <if 语句> ::= **if** '(' <表达式> ')' <语句块> [**else** <语句块>] (注: [] 中的项表示可选)
20. <表达式> ::= <加法表达式> { **relop** <加法表达式> } (注: **relop** -> <|<=>|>=|==|!=>)
21. <加法表达式> ::= <项> { + <项> | - <项> }
22. <项> ::= <因子> { * <因子> | / <因子> }
23. <因子> ::= **num** | '(' <表达式> ')' | <ID> **FTYPE** | <数组>
24. **FTYPE** ::= <call> | 空
25. <call> ::= '(' <实参> ')'
26. <数组> ::= <ID> '[' <表达式> ']' | <数组> '[' <表达式> ']'
27. <实参> ::= <实参列表> | 空

28. $\langle \text{实参列表} \rangle ::= \langle \text{表达式} \rangle \{, \langle \text{表达式} \rangle\}$

29. $\langle \text{ID} \rangle ::= \text{字母} (\text{字母} | \text{数字})^*$

3.1.2 语义分析与中间代码生成

本实验的语义分析在语法分析规约过程中实现，步骤描述如下：

1. **三元组序列进行规约。**利用构造的 LR(1) 分析表对 token 三元组序列进行规约，未规约成功则提示出错，并给出出错定位以及出错信息。
2. **执行子程序，分句子类型进行翻译。**执行相应的语义子程序，完成翻译，生成中间代码。此处根据语句类型进行不同的翻译过程。句子类别主要分为说明语句、简单算数表达式和赋值语句、布尔表达式、控制语句、数组元素引用等。

在实现时将按照每个翻译语句的预先编号进入对应的翻译过程，在每个 case 中对语义栈进行操作，同时将四元式写入到输出文件中。能实现翻译的表达式类型有五类，分别为：

说明语句： 关键字定义变量名性质（如数据类型），文法描述如下：

$$S \rightarrow \text{integer } i$$
$$S \rightarrow \text{real } i$$
$$S \rightarrow S, i$$

算数表达式和赋值语句： 实现含有整形变量的算数表达式，文法描述如下：

$$S \rightarrow i := E$$
$$E \rightarrow E_1 + E_2$$
$$E \rightarrow E_1 \times E_2$$
$$E \rightarrow -E_1$$
$$E \rightarrow (E_1)$$
$$E \rightarrow i$$

布尔表达式： 布尔运算符 and, or, not 关系运算符 小于/小于等于/ 等于/ 不等于/ 大于/大于等于，文法描述如下：

$$E \rightarrow \text{not } E \mid E \text{ and } E \mid E \text{ or } E \mid i \mid i_1 \text{ relop } i_2 \mid (E)$$

控制流语句： if-then / if-then-else / while-do，文法描述如下：

$$S \rightarrow \text{if } E \text{ then } S$$
$$S \rightarrow \text{while } E \text{ do } S$$
$$S \rightarrow \text{begin } L \text{ end}$$
$$S \rightarrow \text{if } E \text{ then } S \text{ else } S$$
$$F \rightarrow F; S$$
$$S \rightarrow A$$
$$F \rightarrow S$$

数组元素引用： 含有数组元素表达式的翻译，文法描述如下：

$\text{Array } A[l_1 : r_1, l_2, r_2, \dots, l_k : r_k] : \text{type};$

3. 执行表达式类型对应的语义子程序完成翻译，生成中间代码。在翻译过程的入口函数切换到语义分析栈，执行出入栈操作，同时将产生式对应的四元式写到四元式输出文件中，供前端读取。

3.1.3 系统呈现

语言与框架 本次实验沿用词法分析与分析实验的程序框架，使用 tkinter 实现用户交互的 UI 界面前端开发⁺。tkinter 是一个用于创建图形用户界面（GUI）的 Python 库，tkinter 可移植性强，可以在不同操作系统稳定运行。同时其提供丰富的功能和组件，如窗口、对话框、按钮、文本框、表格、菜单等，可以满足快速开发编译器展示界面的要求。从语言上来说，与基于 C++ 的 Qt 框架相比，tkinter 具有 Python 语言简洁易读等优点，因此我们选用 tkinter 进行前端呈现。

语法树绘图 语法树绘图沿用实验一中使用的 graphviz 进行绘图，转为 PNG 格式进行展示。该绘图工具需要使用编译器的用户在本地安装 graphviz 并配置好本地环境变量。

用户交互逻辑与系统鲁棒性 在进行界面逻辑设计时，考虑到用户交互的便捷性，在主页面设置用户输入文本框，便于用户对其程序进行调整；由于重要数据结构通过读取文本文件载入，模块调用的先后顺序需要注意。在界面设计中，用户只能够依次点击：文件上传、词法分析、语法分析、语义分析和中间代码生成，对用户点击限制通过将按钮 disable 实现。由此保证了文本文件的读取安全性，防止程序读取到上一次生成的文件结果，增强程序鲁棒性。

3.2 数据类型定义

整个分析过程中均需使用终结符的枚举类 VT、非终结符的枚举类 VN。包括数据类型、括号、运算符、关键字、空格、换行符、NULL 等，分别将其对应一个数字，再定义类似哈希表的结构，将名称与数字进行对应，便于存储与后续处理。数据结构定义如下：

<pre>1. class VT(Enum): 2. # 0 ~ 9 3. VTNull = 0 4. VTInteger = 1 5. VTString = 2 6. VTBoolean = 3 7. 8. VTIdentifier = 83 9. VTWord = 84</pre>	<pre>1. VT_names = [2. "Null", 3. "Integer", 4. "String", 5. "Boolean", 6. 7. "Identifier", 8. "Word" 9.]</pre>
---	---

3.2.1 词法分析

本次词法分析相较于上一次实验，添加了预处理类 Scanner，其主要功能为在进行词法分析之前预先将所有的注释部分去除掉。主要数据结构包括：预处理类 Scanner、分割组成类 LexSegment、字典树类 Trie（字典树结点 Res）、词法分析类 LexAnalyzer。

预处理类 Scanner

```
1. class Scanner:
2.     def __init__(self):
3.         self.proc = Process()
4.         self.FileName = ""
5.         self.code = ""
6.         self.iter = None
7.         self.line = 0
8.
9.         # 对输入字符串进行预处理，去除注释和无用字符
10.    def Preprocessing(self, InputStr, InputStrLen):
11.
12.        # 获取输入文件中的 tokens
13.    def getTokens(self, FileName):
14.
15.        # 处理关键字
16.    def dealKeyWord(self, name, token_type):
17.
18.        # 处理标识符
19.    def dealID(self, name, token_type):
20.
21.        # 处理整数
22.    def dealInteger(self, name, token_type):
23.
24.        # 处理浮点数
25.    def dealFloat(self, name, token_type):
26.
27.        # 处理运算符
28.    def dealOperator(self, name, token_type):
29.
30.        # 处理分隔符
31.    def dealDelimiter(self, name, token_type):
32.
33.        # 处理字符
34.    def dealChar(self, name, token_type):
35.
36.        # 跳过空白字符
37.    def skipBlank(self):
38.
```

```

39.
40.     # 获取下一个 Token
41.     def getNextToken(self):

```

词法分析类 LexAnalyzer 主类

定义 5 个成员：

1. original_code 为原始字符串；
2. code_length 为字符串长度 1
3. inner_result 暂存词法分析结果；
4. component_type 记录枚举类型
5. Get_Result()记录字典树；

对外提供 3 个方法：

1. Analyze()进行词法分析的总过程；
2. Get_Result()传出词法分析的结果；
3. Delete_Annotation()调用 Scanner 类获得删除注释后的结果。

```

1. class LexAnalyzer:
2.     def __init__(self):
3.         self.original_code = None
4.         self.code_length = -1
5.         self.inner_result = []
6.         self.component_type = []
7.         self.trie = Trie()
8.
9.     # 内部函数
10.    # 分析宏指令、字符、字符串、单行注释、多行注释
11.    def pre_analyze(self): ...
12.    # 分析分隔符，如空格和回车
13.    def analyze_spliters(self): ...
14.    # 分析关键字与运算符
15.    def analyze_keywords_and_operators(self): ...
16.    # 分析整数、浮点数、标识符等单词
17.    def analyze_words(self): ...
18.    # 储存分析结果
19.    def set_component(self, start_index, end_index, component): ...
20.
21.    # 对外函数
22.    def Analyze(self, code): ...
23.    def Get_Result(self): ...
24.    def Delete_Annotation(self): ...

```

3.2.2 语法分析

语法分析的主类

对外提供 2 个方法：

1. `getProduction()` 获得输入语句的产生式，并写入文件保存；
2. `getFirst()` 获得 First 集；
3. `getItemSet()` 获得项目集并打印；

写入语法树、最终形成 LR(1) 分析表、以上过程的总控程序被封装在 `BlackBox.py` 黑盒中，故此处没有列出。

```
1. class SyntaxAnalyzer:
2.     def __init__(self):
3.         self.VTs = []
4.         self.VNs = []
5.         # 记录规则的列表
6.         # 第一个元素是变元的标号，之后的元素是该变元推出的表达式
7.         # 如 [-200, -100, 45, 12] 代表 -200 => -100, 45, 12
8.         self.PATs = []
9.         self.InitVTs()
10.        self.InitVNs()
11.        self.InitPATs()
12.        # 初始化终结符集
13.        def InitVTs(self): ...
14.        # 初始化非终结符集
15.        def InitVNs(self): ...
16.        # 初始化规则向量
17.        def InitPATs(self): ...
18.
19.        # 输出 LR(1) 分析表到 LR1Table.txt 文件
20.        def LR1_Analyze_file(self, out): ...
21.
22.        # 以下语法分析函数只给出定义，具体函数的实现将在 4.2 中展开
23.        # 依据格式输入文法
24.        # 前一个参数输入，后三个参数输出
25.        def inputGrammar(file_path, get_num, get_string, production): ...
26.        # 合并左式相同的产生式
27.        # 前两个参数为输入，后一个参数为输出
28.        def getProduction(production, get_num, get_produce): ...
29.        # 通过 DFS 算法计算每个文法符号的 First 集
30.        def dfsGetFirst(production, getNum, getProduce, first, nv, nump, getfirst):
31.            ...
32.        def getFirst(production, getNum, getProduce, first): ...
33.        # 对上一个函数计算得到的 First 集进行去重
34.        # 如 First(A)={a,b,a}, 则需要去重为{a,b}
35.        def change_production(production): ...
36.        def addedge(from1, to, w): ...
```

```

36. # 项目与项目集的运算
37. def itemInSet(a, b): ...
38. def itemSetMerge(a, b):
39. def itemSetEqual(a, b): ...
40. def findItemSet(a, b): ...
41. # 求解项目集闭包
42. def getItemClosure(t, get_num, get_produce, production, first): ...
43. # 获得项目集族
44. def getItemSet(get_num, get_produce, production, first): ...
45. # 打印项目集族
46. def printItemSet(production, getString): ...

```

3.2.2 语义分析和中间代码生成

主要数据结构包括：语义分析栈 sStack、符号表 symbolTable、函数表 funTable、中间变量名序号 curTempId、当前标号名序号 curTableId。这些变量以及相关函数通过 semanticAnalyze 函数统筹规划，主要完成产生式规约时进行语义分析的全过程，并生成对应中间代码。该部分也会调用 3.2.1 中的词法分析类 LexAnalyzer、语法分析类 SyntaxAnalyzer。

语义分析栈的类 sStack

该部分提供一个接口，记录并打印语义分析栈，作为最后展示界面的一个窗口。

```

1. class sStack:
2.     def __init__(self, nump=0, ppos=0, forward=[]):
3.         self.nump = nump
4.         self.ppos = ppos
5.         self.forward = forward
6.
7.     def __eq__(self, other):
8.         return self.nump == other.nump \
9.             and self.ppos == other.ppos \
10.            and set(self.forward) == set(other.forward)
11.
12.     def print_SysStack(self):

```

符号表 symbolTable 类

```

1. class SymbolTableEntry:
2.     def __init__(self, name, type, scope, offset):
3.         self.name = name      # 标识符名称
4.         self.type = type      # 标识符类型
5.         self.scope = scope    # 标识符作用域
6.         self.offset = offset  # 标识符在内存中的偏移量
7.     def insert(self, entry):
8.         self.entries.append(entry)

```



```

9.
10.     def lookup(self, name):
11.         for entry in self.entries:
12.             if entry.name == name:
13.                 return entry
14.         return None

```

函数表 funcTable 类

```

1.  class FunctionTableEntry:
2.     def __init__(self, name, return_type, parameters):
3.         self.name = name           # 函数名称
4.         self.return_type = return_type # 函数返回类型
5.         self.parameters = parameters # 函数参数列表
6.
7.     def insert(self, entry):
8.         self.entries.append(entry)
9.
10.    def lookup(self, name):
11.        for entry in self.entries:
12.            if entry.name == name:
13.                return entry
14.        return None

```

语义分析的主类

对外只提供 1 个方法 `generate_intermediate_code()`，即获得四元式形式的中间代码其他内部函数包括插入符号表和函数表、在符号表和函数表中查找、管理作用域的进入与退出等。

```

1.  class SemanticAnalyzer:
2.     def __init__(self):
3.         self.symbol_table = SymbolTable()
4.         self.function_table = FunctionTable()
5.         self.current_scope = None
6.
7.         # 将标识符插入符号表
8.     def insert_symbol(self, name, type, scope, offset):
9.
10.        # 在符号表中查找标识符
11.    def lookup_symbol(self, name):
12.
13.        # 将函数插入函数表
14.    def insert_function(self, name, return_type, parameters):
15.
16.        # 在函数表中查找函数

```

```
17.     def lookup_function(self, name):
18.
19.         # 对表达式进行类型检查
20.     def type_check(self, expression):
21.
22.         # 管理作用域的进入与退出
23.     def scope_management(self):
24.
25.         # 生成中间代码
26.     def generate_intermediate_code(self):
```

四 详细设计

总函数通过 display.py 文件进行统筹展示，以 tkinter 库作为前端展示工具，将词法分析、语法分析、语义分析模块中的接口转为函数插槽的形式与前端按钮关联起来。以下为整个系统的函数调用关系。

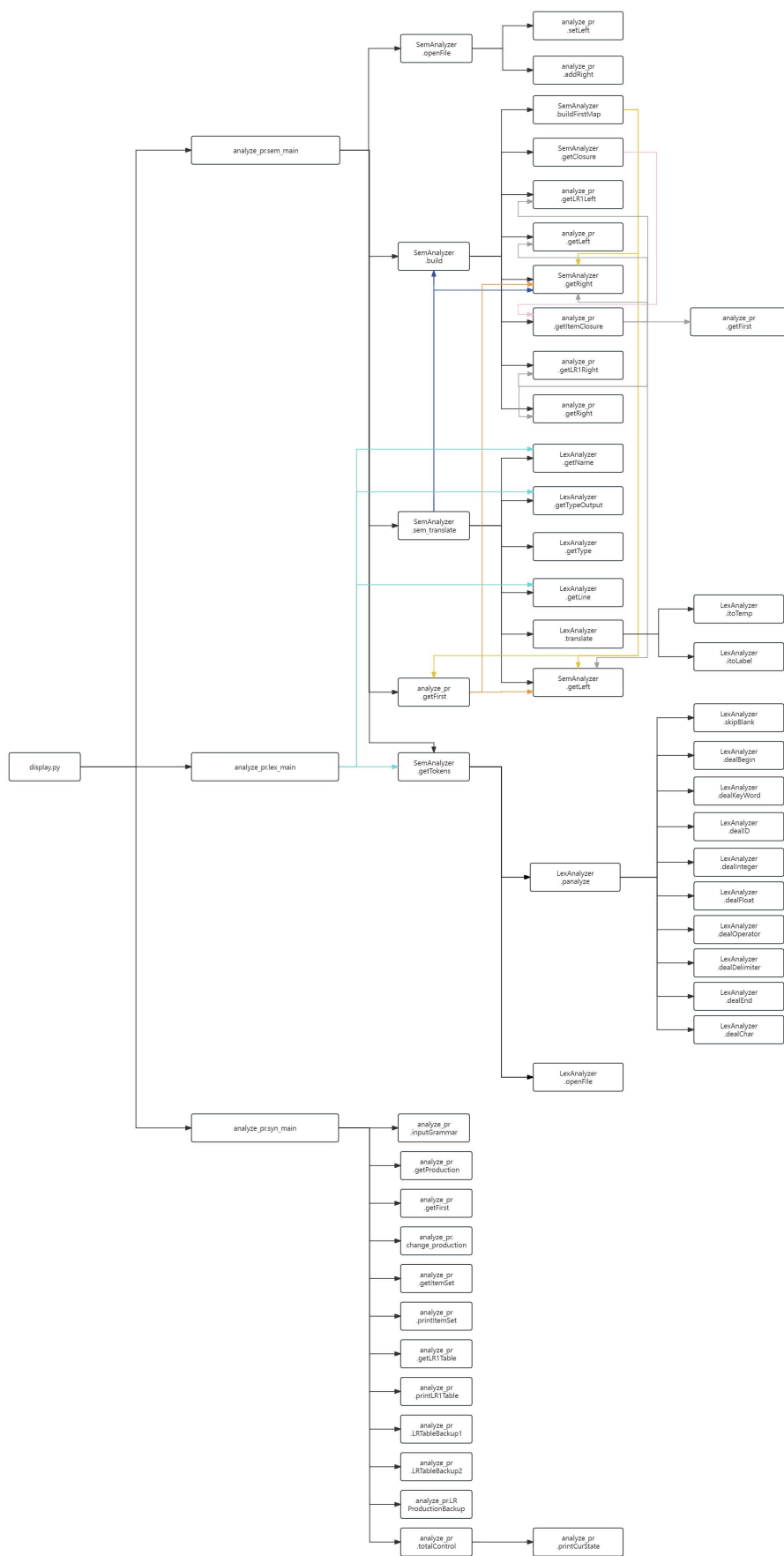


图 11 总体函数调用关系

4.1 词法分析器设计方法

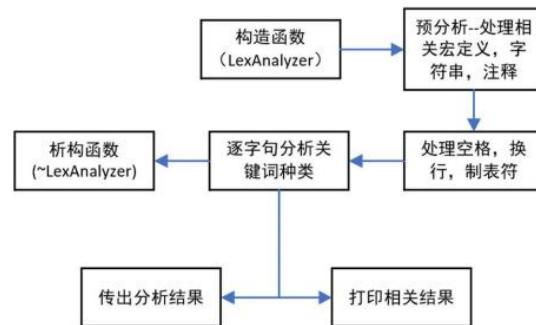


图 11 词法分析的主要流程

4.1.1 LexAnalyzer 类函数

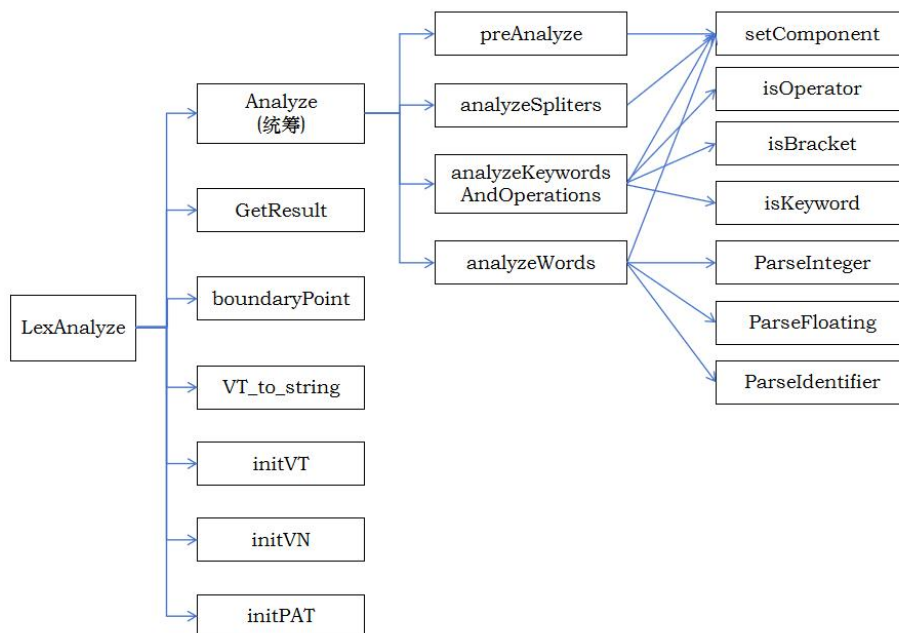


图 12 本作业词法分析部分的函数调用关系示意图

词法分析具体分为 4 步，该部分的函数调用逻辑如上图所示。该部分设计一个函数统筹全过程调度，在 Analyze 函数中分别执行，Analyze 函数内容如下。

```
1. def Analyze(self, code):
2.     self.__init__()
3.     self.original_code = bytearray(code, 'utf-8')
4.     self.code_length = len(self.original_code)
5.     self.component_type = [VT.VTNull for _ in range(self.code_length + 1)]
6.     self.pre_analyze()
7.     self.analyze Splitters()
8.     self.analyze_keywords_and_operators()
9.     self.analyze_words()
```

接下来具体介绍主函数中的几个函数，由于本次作业的重点并不在词法分析上，故此处不再粘贴代码。

Pre_analyze 函数 用于分析字符、字符串、单行注释、多行注释。首先从源程序中去除一些无用的注释以及需要替换的宏指令和一些具有干扰性的字符、字符串，其实现主要由几个状态判断 bool 变量来完成。

Analyze_spliters 用于分析分隔符，这一部分较为简单，主要功能是将源程序中的分割符统一，便于之后其他关键字、标识符等词语的分析。

Analyze_keywords_and_operators 函数分析关键字与运算符。主要采用字典树匹配关键字与运算符，这样可以避免回退，降低时间复杂度。

Analyze_words 分析整数、浮点数、标识符和其他单词。

4.1.2 预处理扫描 panalyze 函数

该函数类的调用流程图如下，功能包括：

- 1.跳过多余的空格与换行
- 2.自动删除注释语句
- 3.输出 token，并在 LexAnalyze 主类中被调用

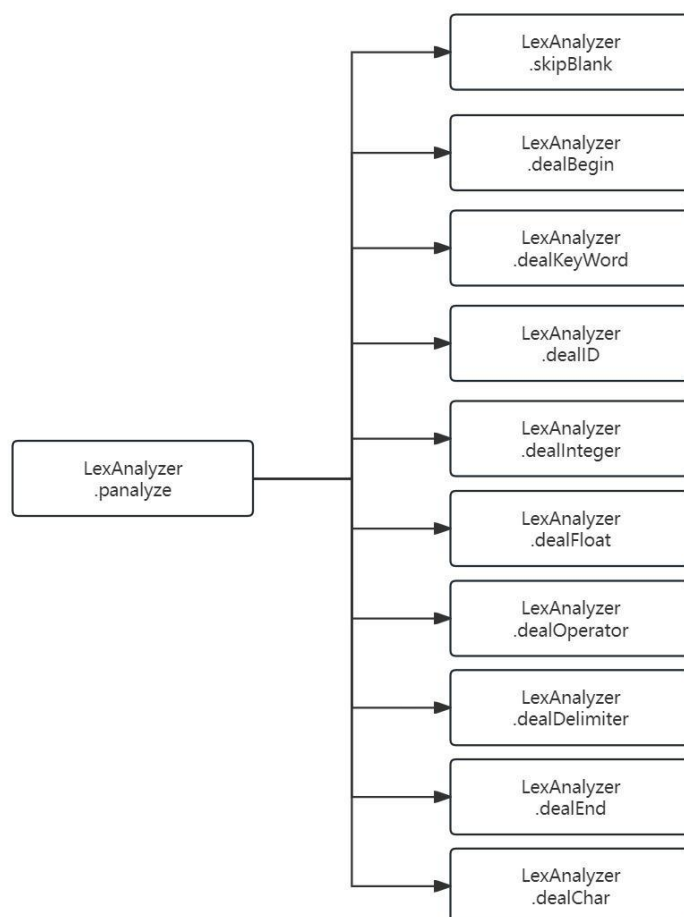


图 13 本作业词法分析部分的 panalyze 扫描预处理函数调用关系示意图

4.1.3 编译预处理 Delete_Annotation 函数

该函数被用于词法分析阶段，用以对输入的源程序进行预处理，具体功能包括：

- 1.对于单行注释（"/"），删除注释符号后的内容，直至遇到回车或换行符；
- 2.对于多行注释（"/* ... */"），删除整个注释块的内容；
- 3.过滤掉无用的字符，仅保留有效的源代码；

在系统读入这部分的时候，首先词法分析阶段调用，确保在进入编译器的词法分析阶段之前，源程序经过了必要的处理，以消除与编译过程无关的内容，从而提高编译器对源代码的准确性和效率。

```

1.  def preprocessing(input_str):
2.      temp_string = ""
3.      i = 0
4.      input_str_len = len(input_str)
5.
6.      while i <= input_str_len:
7.          if i < input_str_len - 1 and input_str[i] == '/' and input_str[i + 1] == '/':
8.              # 若为单行注释"/"，则去除注释后面的内容，直至遇到回车换行
9.              while i < input_str_len and input_str[i] != '\n':
10.                  i += 1 # 向后扫描
11.                  i += 1 # 跳过回车换行
12.          elif i < input_str_len - 1 and input_str[i] == '/' and input_str[i + 1] == '*':
13.              # 若为多行注释"/* ... */"，则去除该内容
14.              i += 2
15.              while i < input_str_len - 1 and (input_str[i] != '*' or input_str[i + 1] != '/'):
16.                  i += 1 # 继续扫描
17.              if input_str[i] == '$':
18.                  print("注释出错，没有找到 */，程序结束!!!")
19.                  exit(0)
20.              i += 2 # 跳过"*/"
21.          elif input_str[i] not in ('\v', '\r'):
22.              # 若出现无用字符，则过滤；否则加载
23.              temp_string += input_str[i]
24.              i += 1
25.
26.      return temp_string

```

4.1.4 Parse_Float, Parse_Identifier, Parse_Integer

本编译器针对浮点数、标识符、整数的识别构造 DFA。在 DFA 转移图如下。由于本次大作页重点是后面的语义分析部分，这里没有再列出代码。

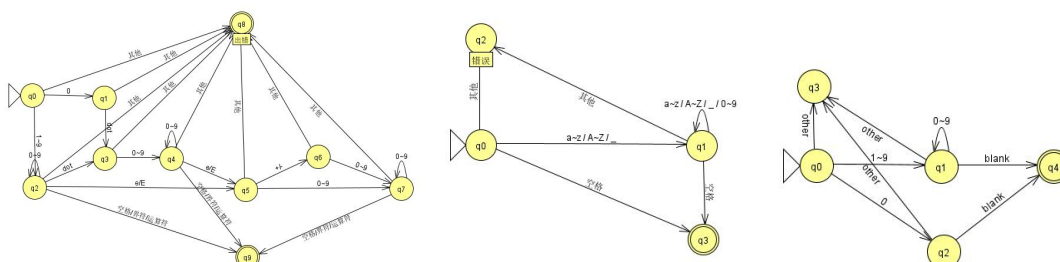


图 14 本作业词法分析部分 DFA 示意图

4.2 语法分析器设计方法

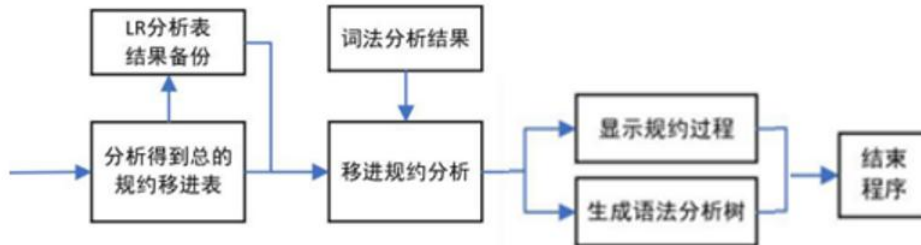


图 17 语法分析器的基本设计思路

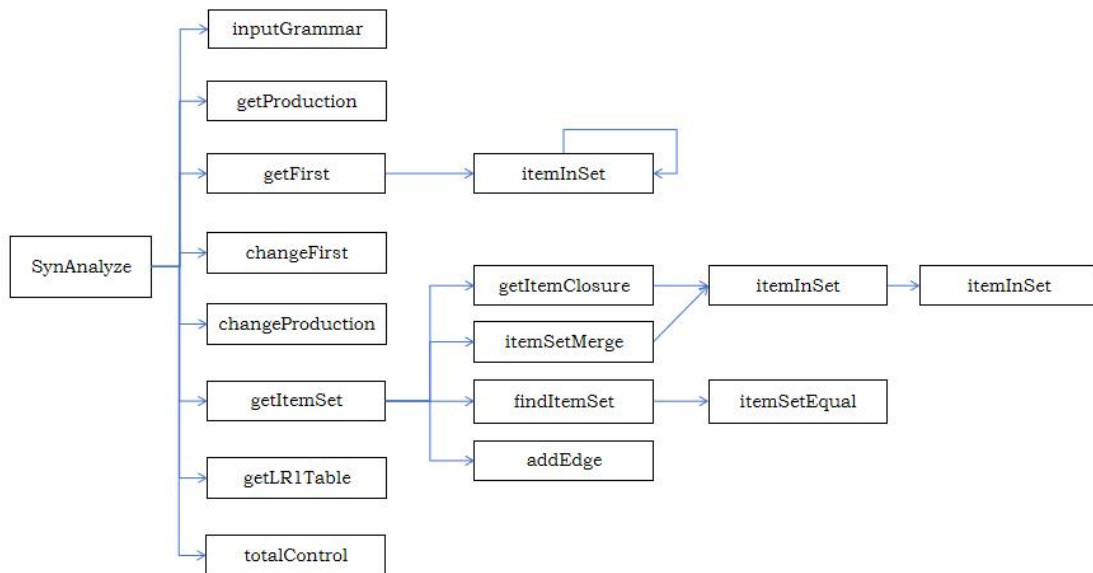


图 18 本作页中的语法分析模块的函数调用分析

4.2.2 获取 First 集

该部分包括 dfsGetFirst、getFirst、changeProduction 共三个函数：dfsGetFirst 用于通过深度优先搜索获得某个式子的所有产生式；getFirst 调用 dfsGetFirst 获取所有终结符的 First 集；changeProduction 用于对 getFirst 生成的结果去重。由于本次作业的重点在于语义分析与中间代码生成，由于与第一次实验重合，则不赘述。

4.2.3 构造项目集闭包

在代码编写中，我们主要采用广度优先搜索算法 BFS 求解项目的闭包，在确定项目的展望字符时，需要用到之前计算的 FIRST 集。主要通过 getItemClosure 函数实现，其功能为求解并构造项目集闭包，由于该部分与第一次实验重合，则不赘述。

4.2.4 构造项目集族

构造有效的 LR (1) 项目集族的办法本质上和构造 LR (0) 项目集规范族的办法是一样的。都需要两个函数 CLOSURE 和 GO，上一小节已经构造出 CLOSURE 函数，因此还需要构造 GO 函数，定义如下：

令 I 是一个项目集， X 是一个文法符号，函数 $GO(I, X)$ 定义为 $GO(I, X) = CLOSURE(J)$ 。关于文法 G 的 LR (1) 项目集族的构造算法为：

```

BEGIN:
   $C := \{Closure([S \rightarrow \bullet S, \#])\};$ 
  REPEAT:
    FOR  $C$  中的每一个项目集  $I$  和  $G$  的每个符号  $X$ 
      IF  $GO(I, X)$  非空且不属于  $C$ , THEN
        把  $GO(I, X)$  加入  $C$  中
  UNTIL  $C$  不再增大
END

```

其具体代码实现如下。该部分包括两个函数：getlItemSet 和 printlItemSet。具体实现代码时，我们仍采用 BFS 算法求解项目集族，实际上最后构造的项目集转移图也是一个 DFA，我们采用记录图的方式来记录这个 DFA。由于与第一次实验重合，则不赘述。

4.2.5 获得 LR(1) 分析表

从文法的 LR (1) 项目集族 C 构造分析表的算法如下：假定 $C = \{I_0, I_1, \dots, I_n\}$ ，令每一个 I_k 的下标 k 为分析表的状态。令那个含有 $[S' \rightarrow \bullet S, \#]$ 的 I_k 的 k 作为分析器的初态。动作 ACTION 和状态转换 GOTO 可构造如下：

- (1) 若项目 $[A \rightarrow \alpha, \beta]$ 属于 I_k ，且 $GOTO(I_k, a) = I_j$ （其中 a 为终结符），则置 $ACTION[k, a]$ 为“把状态 j 和符号 a 移进栈”，简记为“ aj ”。
- (2) 若项目 $[A \rightarrow \alpha, \beta]$ 属于 I_k ，则置 $ACTION[k, a]$ 为“用产生式 $A \rightarrow$ ”规约”，简记为“ rj ”；其中假定 $A \rightarrow$ 为文法 G 的第 j 个产生式。
- (3) 若项目 $[S' \rightarrow S, \#]$ 属于 I_k ，则置 $ACTION[k, \#]$ 为“接受”，简记为“ acc ”
- (4) 若 $GO(I_k, a) = I_j$ ，则置 $GOTO[k, A] = j$ 。

4.2.6 生成语法树

语法树绘制的难点在于语法树的构造，因为 LR (1) 分析是自下而上的分析，而由下而上的构造一棵树是比较困难的，我们的解决办法是在总控程序进行规约的时候，把规约用到的产生式存到一个栈里面，规约成功后再逐个取出，从根节点 S 开始构造语法树。因为 LR (1) 是最左规约的，对应着最右推导，所以每次只需要取最右边的非终结符节点进行推导展开，将该非终结符节点作为根节点构造子树，如此完成最终语法树的构造。最终绘制语法树。

本作也使用 dot 语言和 GraphViz 库生成了比较美观的语法树图片，如下。由于本次大作也的重点并不在语法分析该部分，而且属于语法分析的附加分支，因此不再列出代码。

4.3 语义分析器设计方法

这里仅选取几个重要的语义规则设计进行展示，其中有关过程、函数调用和数组的语义规则将在下几章进行专门介绍。

4.3.1 规则设计

变量声明语句 对应语义规则可以举例为 `declaration.type = int; declaration.size = 4;`其主要流程如下:

- ✓ 使用语义规则获取变量的类型, 大小;
- ✓ 检查符号表中是否存在同名的变量;
- ✓ 若不存在, 创建一个新的数据项, 保存该变量的名称, 类型大小。

赋值语句 对应语义规则可以举例为 `id = expression ; array = expression ;`其主要流程为: 首先查找表达式左端变量是否在符号表中, 不存在则报错; 其次, 判断变量是常数还是标识符; 最后进行中间代码生成。中间代码生成的主要流程如下:

- ✓ 找到赋值语句左端的变量 `s`
- ✓ 新建一个中间变量 `n`
- ✓ 基于表达式 `expression`, 得到对 `n` 赋值的中间代码。
- ✓ 将计算得到的 `n` 的值赋给 `s`。

条件语句 对应语义规则可以举例为 `if ( expression ) block; if ( expression ) block else block;`其主要流程为: 对于 `if ( expression ) block` 语句:

- ✓ 建立真值跳转地址和结束地址
- ✓ 获得 `block` 在状态栈中的内容和 `expression` 在状态栈中的内容
- ✓ 计算条件表达式, 生成相应的中间代码
- ✓ 判断是否表达式是否为真 (为真则设置真值跳转地址并跳转; 否则跳转到结束地址)
- ✓ 生成语句中间代码, 结束后, 设置结束地址

对于 `if ( expression ) block else block` 语句:

- ✓ 建立真值地址, 假地址, 结束地址
- ✓ 获取假植表达式, 真值表达式, 条件表达式
- ✓ 计算条件表达式
- ✓ 和没有 `else` 语句的过程类似, 根据计算结果进行跳转
- ✓ 在真值表达式后加入跳转到结束地址的指令

循环语句 对应语义规则可以举例为 `while ( expression ) block;`其主要流程为:

- ✓ 建立四个分支入口: `true`、`false`、`begin`、`end`
- ✓ 获取循环内容状态
- ✓ 获取条件表达式
- ✓ 跳转到 `begin` 地址
- ✓ 计算条件表达式并给出中间代码
- ✓ 若表达式为真, 跳转到 `true` 地址; 为假, 跳转到 `false` 地址
- ✓ 生成循环内容的中间代码 (`break` 则跳转到 `false` 地址, `continue` 则跳转到 `true` 地址)
- ✓ 最后, 跳转到 `begin` 地址并设置 `false` 地址

具体的语义规则如下:

程序结构及主函数声明:

```
1. [0] program_ -> program
2. [1] program -> main_declaration
```

3. [2] program -> declaration_list main_declaration
4. [3] main_declaration -> type_specifier ID () compound_statement

类型标识符翻译:

1. [4] type_specifier -> char
2. [5] type_specifier -> int
3. [6] type_specifier -> float

复合语句翻译:

1. [7] compound_statement -> { }
2. [8] compound_statement -> { declaration_list }
3. [9] compound_statement -> { statement_list }
4. [10] compound_statement -> { declaration_list statement_list }

声明列表及语句列表翻译:

1. [11] declaration_list -> declaration_type
2. [12] declaration_list -> declaration_list declaration_type
3. [13] declaration_type -> type_specifier ID ;
4. [14] statement_list -> statement
5. [15] statement_list -> statement_list statement

声明语句和表达式语句:

1. [16] statement -> compound_statement
2. [17] statement -> expression_statement
3. [18] statement -> iteration_statement
4. [19] statement -> jump_statement
5. [20] statement -> selection_statement
6. [21] expression_statement -> ;
7. [22] expression_statement -> expression ;
8. [23] expression -> assignment_expression

赋值表达式:

1. [24] assignment_expression -> ID = assignment_expression
2. [25] assignment_expression -> logical_or_expression

逻辑表达式:

1. [26] logical_or_expression -> logical_and_expression
2. [27] logical_or_expression -> logical_or_expression || logical_and_expression
n
3. [28] logical_and_expression -> equality_expression
4. [29] logical_and_expression -> logical_and_expression && equality_expression

相等表达式翻译:

1. [30] equality_expression -> relational_expression
2. [31] equality_expression -> equality_expression == relational_expression
3. [32] equality_expression -> equality_expression != relational_expression

关系表达式翻译:

1. [33] relational_expression -> additive_expression
2. [34] relational_expression -> relational_expression < additive_expression
3. [35] relational_expression -> relational_expression > additive_expression
4. [36] relational_expression -> relational_expression <= additive_expression
5. [37] relational_expression -> relational_expression >= additive_expression

运算表达式与一元表达式:

1. [38] additive_expression -> multiplicative_expression
2. [39] additive_expression -> additive_expression + multiplicative_expression
3. [40] additive_expression -> additive_expression - multiplicative_expression
4. [41] multiplicative_expression -> unary_expression
5. [42] multiplicative_expression -> multiplicative_expression * unary_expression
6. [43] multiplicative_expression -> multiplicative_expression / unary_expression
7. [44] unary_expression -> ID
8. [45] unary_expression -> INT
9. [46] unary_expression -> FLOAT
10. [47] unary_expression -> CHAR
11. [48] unary_expression -> (logical_or_expression)

循环、跳转、选择翻译:

1. [49] iteration_statement -> while (expression) compound_statement
2. [50] jump_statement -> continue ;
3. [51] jump_statement -> break ;
4. [52] selection_statement -> if (expression) statement
5. [53] selection_statement -> if (expression) statement else statement

4.3.2 主要函数设计

语义栈弹出函数 stack_pop()

语义分析过程中弹出返回语义栈顶部元素。

1. def stack_pop():
2. """

```

3. 从语义栈中弹出并返回顶部元素
4. """
5. var = varStack[-1] # 获取顶部元素
6. varStack.pop()    # 弹出顶部元素
7. return var

```

字符栈弹出函数 str_stack_pop()

语义分析过程中弹出返回字符栈顶部元素。

```

1. def str_stack_pop():
2.     """
3.     从 strStack 弹出并返回顶部元素
4.     """
5.     string = strStack[-1] # 获取顶部元素
6.     strStack.pop()        # 弹出顶部元素
7.     return string

```

具体语义分析总控翻译过程 sem_translate()

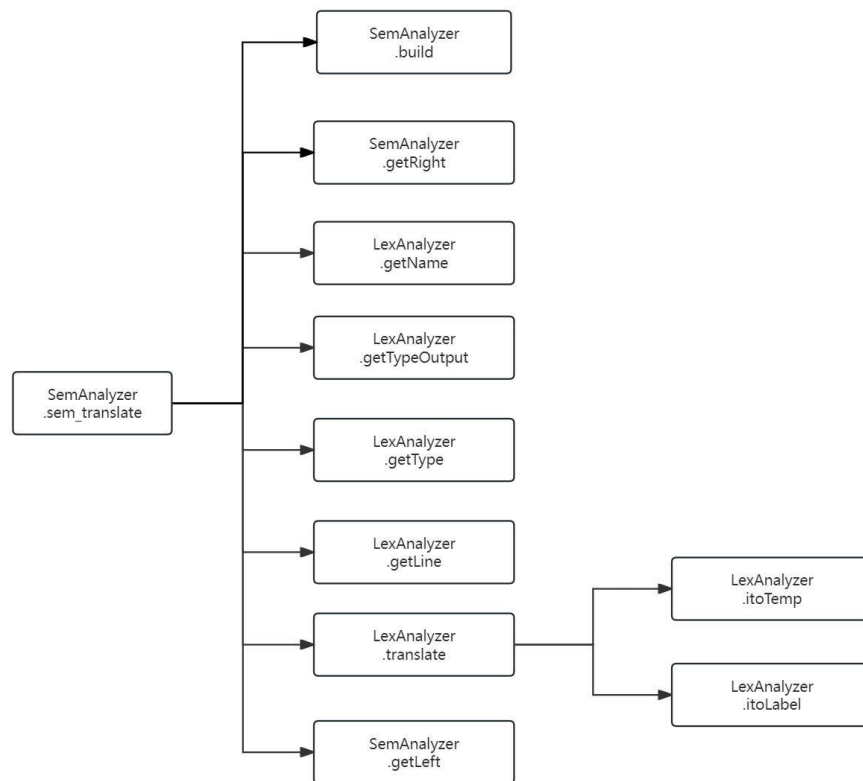


图 20 本作业语义分析部分总控函数的调用关系示意图

如上图所示的函数调用分析，在执行语法分析的过程中，通过 sem_translate 函数统筹执行相应的语义子程序。完成翻译，生成中间代码。执行不同表达式对应的子程序进行翻

译，将语法分析中的分析栈改为对应的语义栈，对栈中元素进行处理，得到表达式及中间代码生成过程并输出。

```
1. def sem_translate():
2.     if len(closuremap) != 0:
3.         return
4.     buildFirstMap()
5.     closure0 = getClosure(LR1item(LR0item(0, 0), "$"))
6.     q = deque()
7.     q.append(closure0)
8.     closurelist.append(closure0)
9.     closuremap[closure0] = len(closuremap)
10.    while q:
11.        lr1set = q.popleft()
12.        transfer.append([])
13.        for variable in variableSet:
14.            newset = set()
15.            for lr1 in lr1set:
16.                lr0 = lr1.getLeft()
17.                id = lr0.getLeft()
18.                pointpos = lr0.getRight()
19.                if build(grammar[id].getRight()) > pointpos and grammar[id].
                    getRight()[pointpos] == variable:
20.                    newset.add(LR1item(LR0item(id, pointpos + 1), lr1.getRig
                        ht()))
21.            if newset:
22.                sem_getRight(newset)
23.                if newset not in closuremap:
24.                    closurelist.append(newset)
25.                    closuremap[newset] = len(closuremap)
26.                    q.append(newset)
27.                    transfer[-1].append((variable, closuremap[newset]))
28.            for terminal in terminalSet:
29.                newset = set()
30.                for lr1 in lr1set:
31.                    lr0 = lr1.getLeft()
32.                    id = lr0.getLeft()
33.                    pointpos = lr0.getRight()
34.                    if len(grammar[id].getRight()) != pointpos and grammar[id].g
                        etRight()[pointpos] == terminal:
35.                        newset.add(LR1item(LR0item(id, pointpos + 1), lr1.getRig
                            ht()))
36.            if newset:
37.                sem_getClosure(newset)
38.            if newset not in closuremap:
```

```

39.         closurelist.append(newset)
40.         closuremap[newset] = len(closuremap)
41.         q.append(newset)
42.         transfer[-1].append((terminal, closuremap[newset]))
43.
44.     for i in range(len(closurelist)):
45.         action.append({})
46.         go.append({})
47.
48.     for lr1 in closurelist[i]:
49.         point = lr1.getLeft().getRight()
50.         right = grammar[lr1.getLeft().getLeft()].getRight()
51.         if point < len(right) and right[point] in terminalSet:
52.             for go_pair in transfer[i]:
53.                 if go_pair[0] == right[point]:
54.                     if right[point] in action[i] and action[i][right[point]]
!= ("S", go_pair[1]):
55.
56.                         print(" old={}".format(action[i][right[point]]))
57.                         action[i][right[point]] = ("S", go_pair[1])
58.
59.     for tf in transfer[i]:
60.         if tf[0] in variableSet:
61.             if tf[0] in go[i]:
62.                 # error
63.                 print("error at go[{}][{}]".format(i, tf[0]))
64.                 go[i][tf[0]] = tf[1]
65.
66.     for lr1 in closurelist[i]:
67.         lookahead = lr1.getRight()
68.         lr0 = lr1.getLeft()
69.         id = lr0.getLeft()
70.         point = lr0.getRight()
71.         right = grammar[id].getRight()
72.
73.         if point == len(right):
74.             if lookahead in action[i] and action[i][lookahead] != ("r", id):
75.                 # error 2
76.                 print("error 2 at action[{}][{}]=r{} old={}".format(i, looka
head, id, action[i][lookahead]))
77.                 # deal with if else / if
78.                 if lookahead not in action[i] or action[i][lookahead]==("r",id):
79.                     action[i][lookahead] = ("r", id)
80.

```

```
81.         if lookahead == "$" and id == 0:
82.             action[i][lookahead] = ("acc", 0)
```

字符栈弹出函数 sem_output()

作业中以四元式的形式输出中间代码、输出语义分析栈内容，并将其写入文件的函数如下。

```
1. def sem_output(tokens):
2.     tempNum = 0
3.     labelNum = 0
4.     sem_build()
5.     # 输出文件重定向
6.     fout = open("SysStack.txt", "w")
7.     if not fout:
8.         print("output open failed")
9.         exit(-1)
10.    print("Symbol stack information:")
11.    # 在此输出到文件
12.    fout.write("Symbol stack information:\n")
13.    st = deque()
14.    st.append((0, "$"))
15.    iter = iter(tokens)
16.    # 在这里输出程序运行中插入 token 的信息
17.    print("Inserted element:")
18.    token = next(iter)
19.    print(token.getName(), token.getTypeOutput())
20.
21.    # 在此输出到文件
22.    fout.write("Inserted element:\n")
23.    fout.write(token.getName() + " " + token.getTypeOutput() + "\n")
24.    while True:
25.        I = st[-1][0]
26.        if token.getType() == TokenType.ID or token.getType() == TokenType.CHAR or token.getType() == TokenType.INT or token.getType() == TokenType.FLOAT:
27.            type = TokenDict[token.getType()]
28.        else:
29.            type = token.getName()
30.
31.        if type in action[I]:
32.            act = action[I][type]
33.            if act[0] == "S":
34.                if token.getType() == TokenType.ID or token.getType() == TokenType.CHAR \
35.                    or token.getType() == TokenType.INT or token.getType() == TokenType.FLOAT:
```

```

36.         varStack.append(token.getName())
37.
38.         st.append((act[1], type))
39.         # 在这里输出程序运行中栈的信息
40.         print("Stack Information:")
41.         for item in reversed(st):
42.             print(item[0], item[1])
43.
44.         # 在此输出到文件
45.         fout.write("Stack Information:\n")
46.         for item in reversed(st):
47.             fout.write(str(item[0]) + " " + item[1] + "\n")
48.         fout.write("\n")
49.         token = next(iter)
50.         # 在这里输出程序运行中插入 token 的信息
51.         print("Inserted element:")
52.         print(token.getName(), token.getTypeOutput())
53.
54.         # 在此输出到文件
55.         fout.write("Inserted element:\n")
56.         fout.write(token.getName()+" "+token.getTypeOutput()+"\n")
57.     elif act[0] == "r":
58.         id = act[1]
59.         right = grammar[id].getRight()
60.         for i in range(len(right)):
61.             if right[i] != "@":
62.                 st.pop()
63.                 newI = st[-1][0]
64.                 st.append((go[newI][grammar[id].getLeft()], grammar[id].getLeft()))
65.         if not translate(id, token.getName()):
66.             print("ERROR! at line", token.getLine())
67.             return token.getLine()
68.     elif act[0] == "acc":
69.         # 关闭 SysStack.txt 文件
70.         fout.close()
71.         fout = open("Quaternion_information.txt", "w")
72.         if not fout:
73.             print("output open failed")
74.             exit(-1)
75.         print("Accept!")
76.         # 输出到文件
77.         fout.write("Accept!\n")
78.         # 在这里输出最后的四元式

```



```
79.         print("Quaternion information:")
80.         print(strStack[-1])
81.         # 输出到文件
82.         fout.write("Quaternion information:\n")
83.         fout.write(strStack[-1] + "\n")
84.         fout.close()
```

五 算法分析框图

5.1 总程序算法流程图

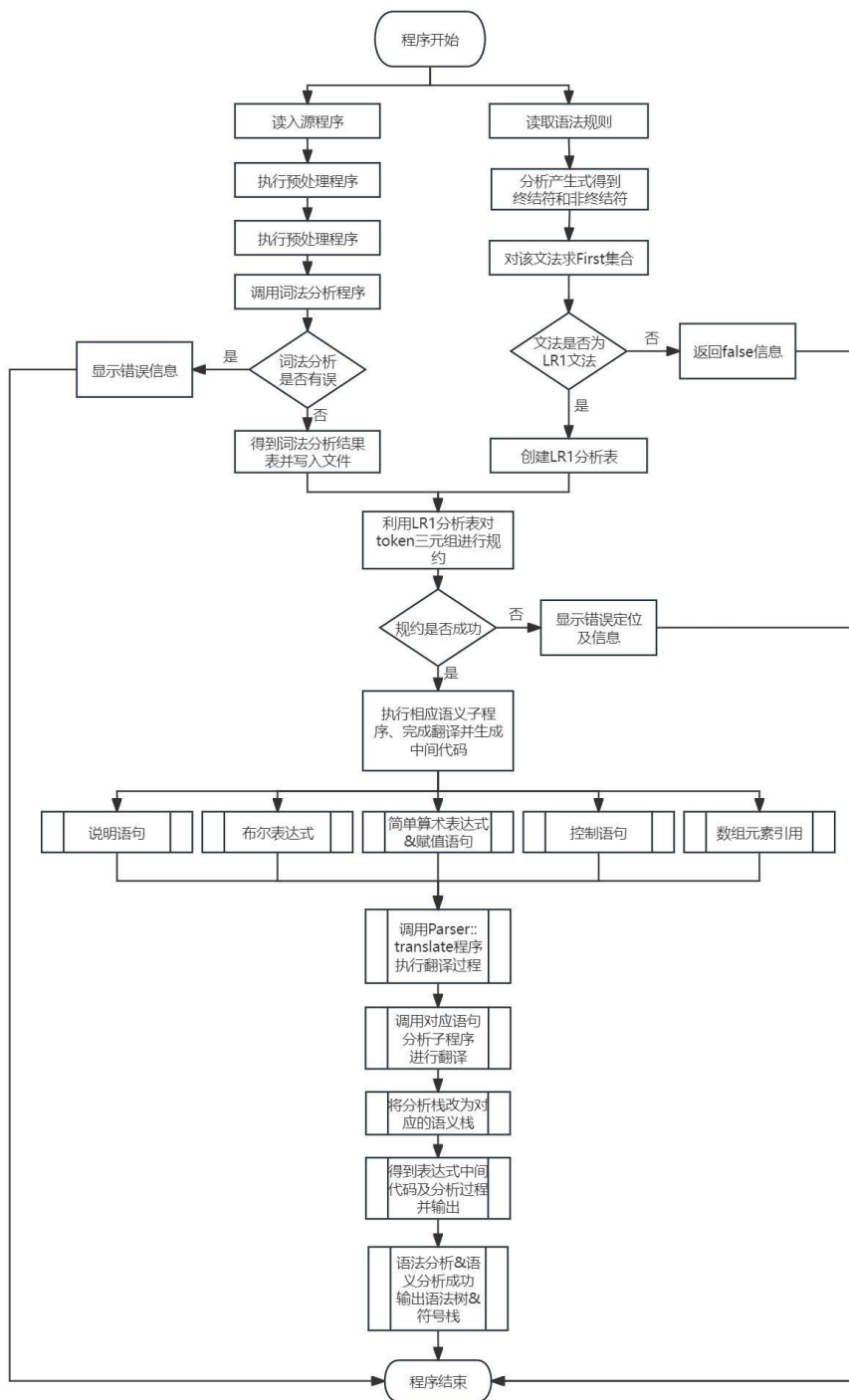


图 22 本作业总程序的算法流程示意图

5.2 词法分析算法流程图

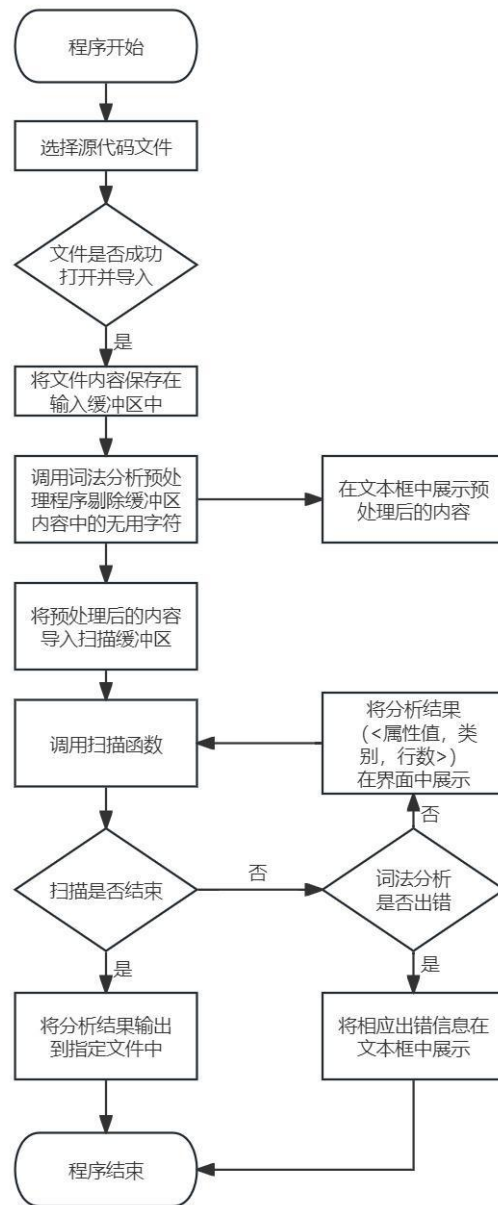


图 23 本作业词法分析部分算法流程示意图

5.3 语法分析算法流程图

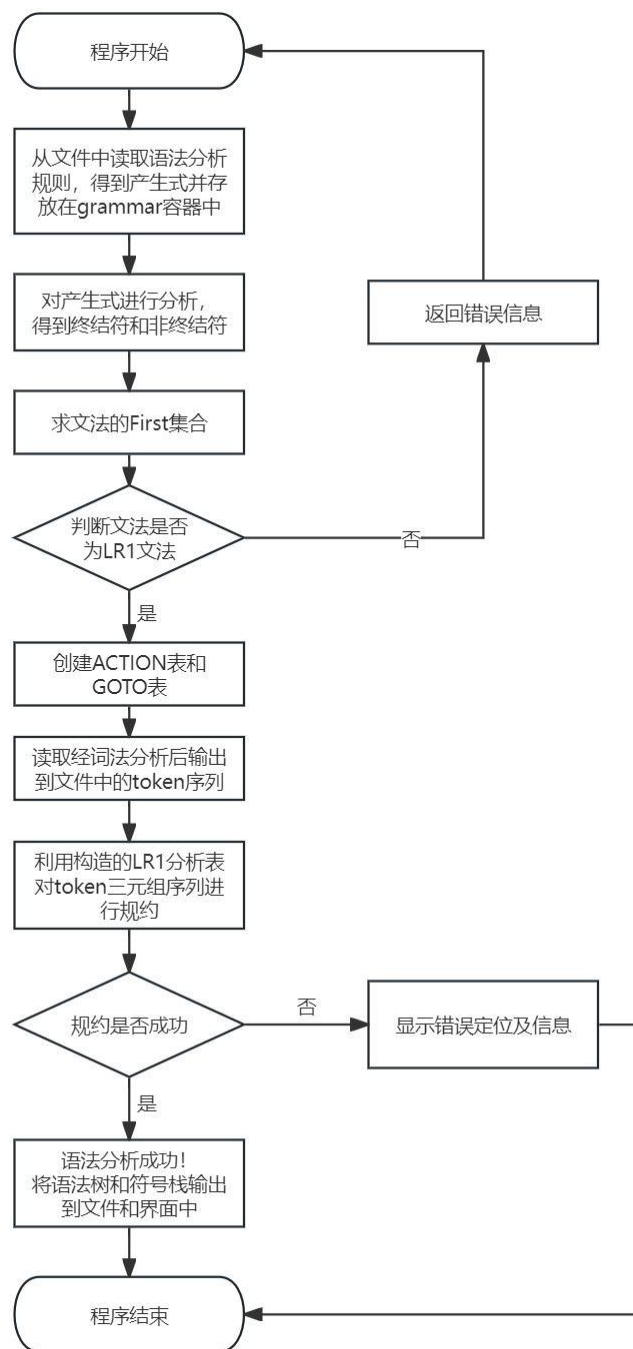


图 24 本作业语法分析部分算法流程示意图

5.4 语义分析及中间代码算法流程图

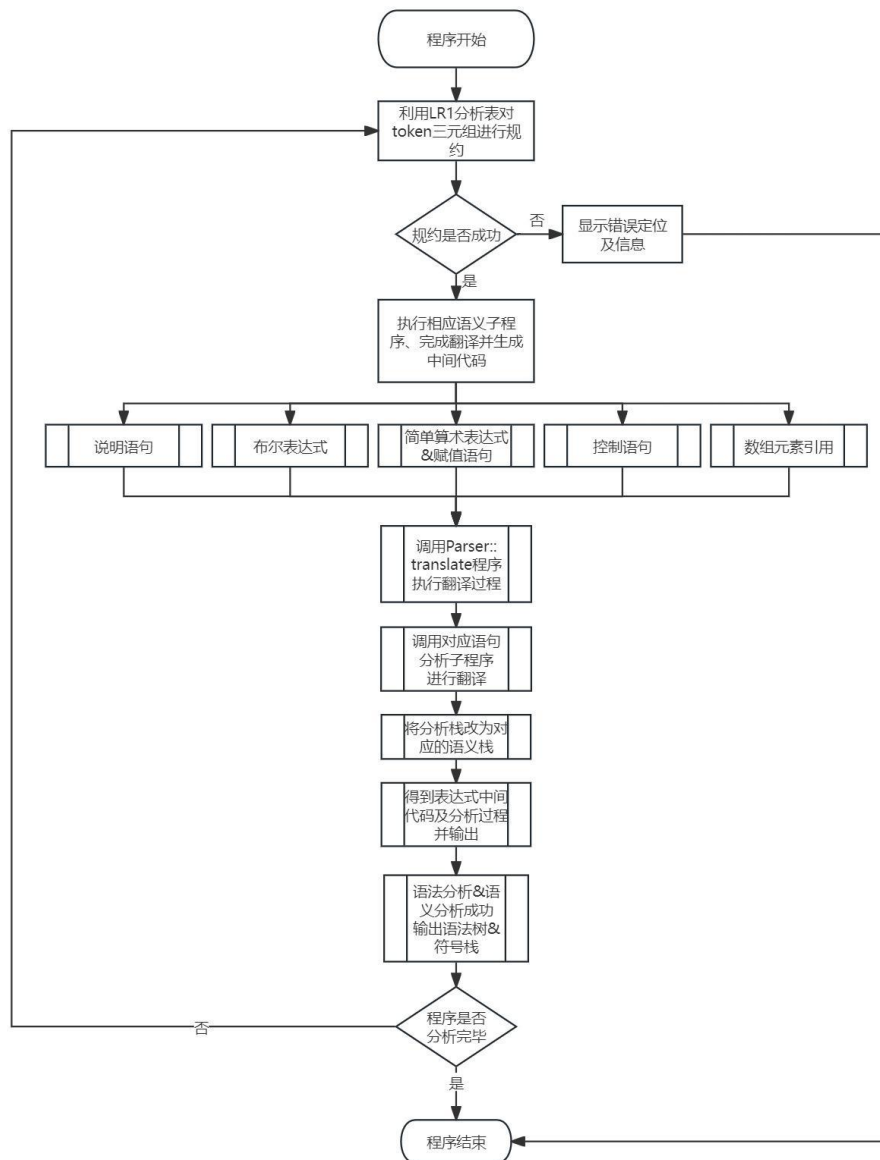


图 25 本作业语义分析及中间代码生成部分算法流程示意图

六 调试分析与错误处理

6.1 调试测试

6.1.1 测试正确文件

测试文件：

```

1. int main(void)
2. {
3.     int b;
4.     int a;

```

```

5.  a = 0;
6.  b = 2.0+4*7;
7.  while ( a > 3)
8.  {
9.      a = a + 1;
10.     b = b + a;
11. }
12. if(a <= b)
13. {
14.     a = 1+2*3-6/3+2*(3+4);
15.     b = a + 2;
16. }
17. else
18. {
19.     b = 9;
20.     a = a*b + b+(0-8*a);
21. }
22. }

```

初始化界面：



文件上传：

词法分析器与语法分析器

```
int main(void)
{
    int b;
    int a;
    a = 0;
    b = 2.0+4*7;
    while ( a > 3)
    {
        a = a + 1;
        b = b + a;
    }
    if(a <= b)
    {
        a = 1+2*3-6/3+2*(3+4);
        b = a + 2;
    }
    else
    {
        b = 9;
        a = a*b + b+(0-8*a);
    }
}
```

文件上传 词法分析 语法分析 中间代码生成

词法分析：

Lexical Analysis Results

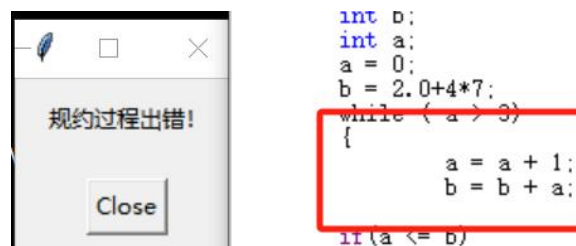
Token	Type
int	int
main	Identifier
(	(
void	void
)	)
{	{
int	int
b	Identifier
;	;
int	int

语法分析：

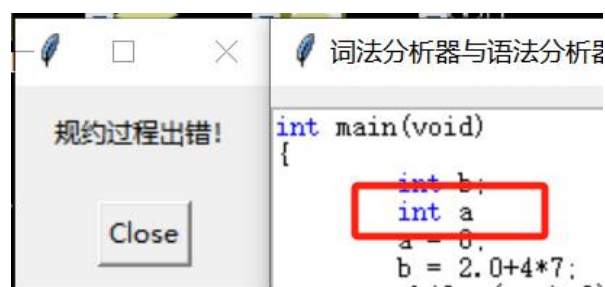
LR1 分析表、规约过程、语法树

6. }

错误 1: 6.1.1 中 while 过程快未闭合

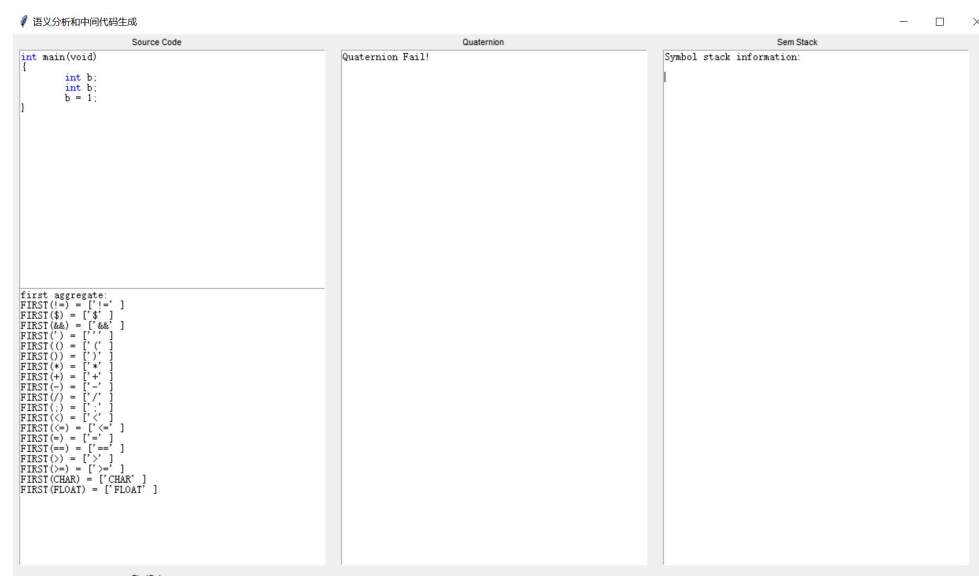


错误 2: 缺少分号



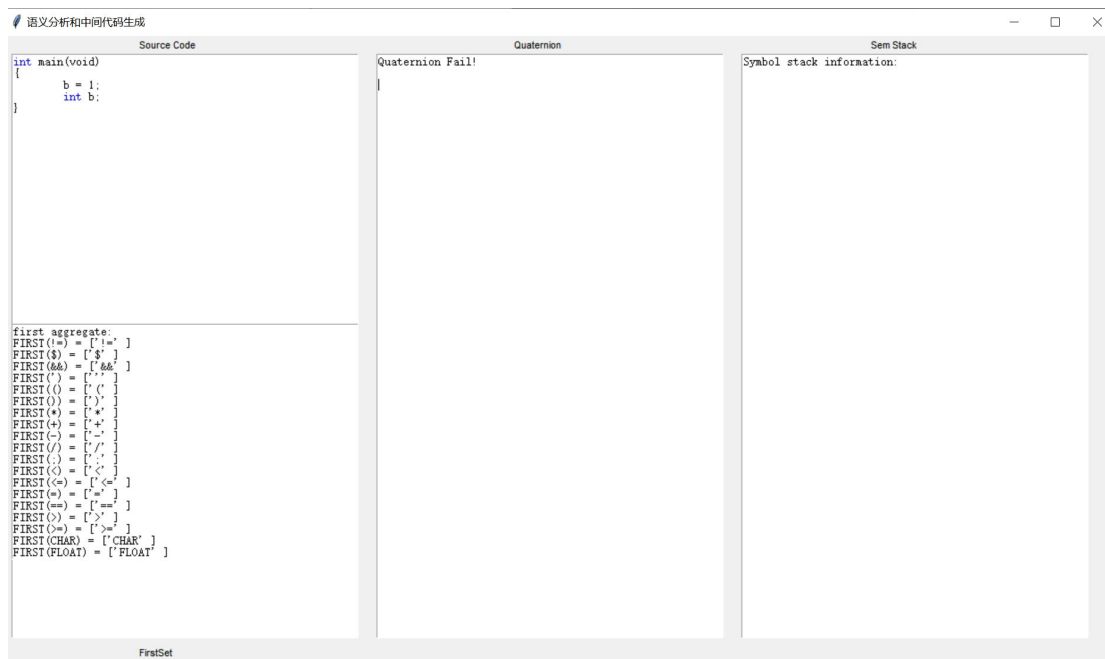
6.1.3 测试静态语义错误文件

错误 1: 变量重定义



变量重定义语义错误, 生成四元式失败

错误 2: 变量使用在声明之前



变量使用在声明之前语义错误，生成四元式失败

6.2 问题发现与解决

6.2.1 字典树的应用

在进行词法分析的过程中，各类字符、关键字等每次读取时都需要进行匹配，速度较慢。联想到数据结构课上所学，我们在这方面应用字典树来提高查找效率，不用每读入一个字符都在列表里查匹配，只要判断是否到达叶结点即可。应用字典树后，分析关键字与运算符过程中减少了无谓字符串的比较，避免了字符串匹配的回退，使得关键字匹配的效率大大提高。

不过字典树属于“以空间换时间”的数据结构，由于一般来说单个文件中的代码行数不会上千上万，因此本处使用字典树是合理的。

6.2.2 缓存思想引入

由于本作业中存在模块间的互相调用，如语法分析会调用词法分析模块，前端呈现将调用两大分析模块。此外，在语法分析内部，光产生式就可能有超过一千条，状态集更是超过一千五百个，整个分析过程十分缓慢。

考虑到对于同样的语法规则，构建得到的 Action Goto 表应该是相同的。而对于相对复杂的语法规则，构建 Action Goto 表涉及的查询次数非常多。于是在构建 First 集的过程中，我们选择在单次构建后，将该表存储到本地文件。后续启动时，首先尝试加载缓存，仅当加载失败才重新构建，以此实现瞬间启动。

6.2.3 文法产生式的空串处理

在预先设计时我们并未考虑到空串的情况，但实际测试的时候发现 ϵ 其实是一个比较头疼的问题，如果输入的产生式里面包含空产生式的话，对于求 FIRST 集和求项目集来说都是一个头疼的问题。

我们采用的处理方法是:求 FIRST 集时 ϵ 显式表示,即求得的 FIRST 集可能包含 ϵ ,而在求项目集族的时候 ϵ 隐式表示,即空产生式从一开始就是一个规约项目,应该直接表示成 “S- $\rightarrow$.” 而不是 “S- $\rightarrow$. ϵ ”,因为最后根据算法生成项目之间的转移关系图是一个 DFA,不可能有空转移的存在。最后经过实际测试,这种处理方法也是完全可行的。

6.2.4 保护栈对象出错

本作业中的语义分析栈是基于上一份大作业中的语发分析栈改动的,对栈中元素进行处理得到表达式及中间代码生成过程并输出。由于在具体的语义分析中,分析栈会有频繁的入栈出栈操作。栈指针指向错误或在取分析栈中的符号序列匹配产生式时错误地将其弹出,最终都会导致分析出错。

当然在上次大作业中,我们还遇到了因没有正确地释放或销毁栈对象导致出错的问题,本次作业中吸取了相关教训。

6.2.5 中间代码局部错误

调试过程中我们还遇到了中间代码局部出错的问题,在逐句调试查找原因后我们将问题定位在了四元式生成中的运算符优先级上。代码中使用了括号算数赋值语句,使得语义分析时某些部分对应的处理操作符和操作数的入栈顺序出现问题,导致该段四元式出错,而作为媒介的四元式与中间代码是相对应的,导致了中间代码局部出错的问题。

在调试错误的过程中,由于各个模块需要注意的细节较多,实现起来比较复杂,因此错误调试,没有形成系统的报错处理机制,这个是需要后期进一步打磨的,同时遇到这些问题也增加了我们对一个完整系统设计的经验。在问题调试中也加强了我们定位 bug 并修复的能力。

七 静态语义错误的诊断和处理

静态语义错误常见类型有:变量未声明、类型不匹配、函数参数类型不匹配、重复声明等。在本实验实现的语义分析及中间代码生成器中,我们实现了变量名重定义、数组越界检查和变量声明检查的静态语义错误检查过程。

7.1 变量重定义

错误分析:

变量名在经过定义后重新再被定义,即数据类型标识符和 identifier 完全相同且出现在同一段程序中。检查过程可对维护的变量栈遍历检查实现。

检测思路:

1. 维护符号栈和变量栈,在变量进行入栈操作之前,检测改变量是否已经出现在变量栈中

出现。

2. 若找到相同的变量名则说明变量已经经过定义，转向报错出口。

代码实现：

```
1. # ...
2. if type in action[I]:
3.     act = action[I][type]
4.     if act[0] == "S":
5.         if iter.getType() in [TokenType.ID, TokenType.CHAR, TokenType.INT, TokenType.FLOAT]:
6.             # 在这里检查变量是否已经在 varStack 中出现，即重复定义
7.             if iter.getName() in varStack:
8.                 print(f"ERROR at line {iter.getLine()}")
9.                 return iter.getLine()
10.            varStack.append(iter.getName())
11.        # ...
12.    # ...
```

作用域问题：

作用域不同时，直接遍历检查当前变量栈中的所有同名变量会造成检测到外部作用域中的同名变量问题。此时可以在每次进入新的作用域时，向变量栈中压入新的符号表。在离开作用域时将相应的符号表出栈，则不会出现检测到外部作用域的同名变量问题。

7.2 数组使用越界

错误分析：

进行数组元素引用时，给出的数组下标超过预先声明的数组下标。

检测思路：

使用数组元素处进行索引位置与声明数组大小比较即可，若超出声明大小范围则进入报错部分。

代码实现：

```
1. # ...
2.
3. elif act.first == "S":
```

```

4.     if iter.getType() == TokenType.ID:
5.         var_info = symbol_table.lookup(iter.getName())
6.
7.         if var_info.is_array:
8.             array_size = var_info.array_size
9.             index = evaluate_expression(iter.get_index_expression())
10.
11.            # 检查数组越界
12.            if index < 0 or index >= array_size:
13.                print(f"ERROR! at line {iter.line}")
14.                return iter.line
15.
16.        if iter.getType() in [TokenType.ID, TokenType.CHAR, TokenType.INT, Token
            Type.FLOAT]:
17.            var_stack.append(iter.getName())
18.        st.append((act.second, type))

```

在检查时，首先检查数组数据类型是否匹配，接着计算声明数组大小并从表达式中提取出所引用的数组下标，进行大小比较即可。

其中从 expression 中提取数组元素下标的函数 evaluate_expression 实现为：

```

1. def extract_array_index(expression):
2.     tokens = parse_expression(expression)
3.     index_tokens = []
4.
5.     for token in tokens:
6.         if token.token_type == TokenType.ID and '[' in token.value:
7.             index_str = token.value.split('[')[1].strip(']')
8.             index_tokens.extend(parse_expression(index_str))
9.
10.    return index_tokens

```

7.3 变量在声明之前使用

错误分析：

变量在声明之前使用指变量在定义之前被用作运算的操作数执行运算、赋值等操作。

检测思路：

1. 在语法分析中识别并提取表达式中的各个变量，备变量声明检查使用
2. 遍历查找符号表，检查判断当前变量是否已经被声明
3. 若未经声明则说明当前使用的变量没有经过定义，转向报错语句

代码实现：

在代码主循环中对变量声明进行检查。查找响应的符号表，判断当前变量是否声明，若报错则转向报错语句进行输出：

```
1. if type in action[I]:
2.     # ...
3.     if act[0] == "S":
4.         if iter.getType() in [TokenType.ID, TokenType.CHAR, TokenType.INT, T
           okenType.FLOAT]:
5.             varStack.append(iter.getName())
6.             st.append((act[1], type))
7.         # ...
8.     else:
9.         print(f"ERROR! at line {iter.getLine()}")
10.    return iter.getLine()
```

八 高级语言的语义检查与中间代码生成

8.1 类型检查：

完整的高级语言具有更为严格的类型系统，在编译过程中需要对变量类型、操作数类型、参数类型和函数返回值类型等进行变量匹配检查。

我们以 C++ 为例，其中常见的类型检查有：

1. **变量声明和使用类型检查：**即在使用时如赋值等操作需要保证与前文声明的数据类型相同。
2. **函数调用参数类型匹配，**即在向函数传参的过程中要确保传入参数类型与函数声明时形参列表声明的参数类型相同。
3. **运算符的操作数类型一致，**例如加法运算要求两侧的操作数至少有一个是数值类型

类型检查分类：类型检查主要检查到两种不同的类型错误，分别为：不兼容的赋值或不兼容

的函数调用。分别对应赋值类型不兼容和函数调用参数类型不兼容两种情况，比如在 C++ 中将字符串赋值给 char 类型变量，或者在使用不同类的实例化初始化一个变量时无法找到两个类的公共父类等等。在其他语言中，如 Java 的 switch 等会存在不同的语义检查机制，如下图所示：

<pre>char x = 'a'; x = "abc"; warning: incompatible pointer to integer conversion assigning to 'char' from 'char [4]' x = "abc"; ^~~~~~</pre>	<pre>switch (true) { case false: break; } error: incompatible types: boolean cannot be converted to int</pre>	<pre>class C { } class D : C { } let x = true ? C() : D() error: result values in '?' expression have mismatching types 'C' and 'D'</pre>
--	--	--

图 26 C++, Java, Swift 类型检查

所有类型错误均需要在语义分析的类型检查中被检测到并进行错误返回。

类型推导：类似的概念应用于 C++ 的泛型模板当中，也就是类型推导找类型错误的方法是要看程序片段能否有一个类型，而此类型并不事先标注好的。即在 C++ 中使用 auto 关键字进行泛型编程

解决方案：实现类型检查需要同时考虑到类型检查和类型推导。类型检查其一般是递归下降实现的，在类型检查器中传入两个参数：抽象语法树 AST 和目标类型。检查过程就是遍历当前语法树(其实为原语法树其中的一个节点，而此时我们把这棵子树抽离出来当作根节点)，最后在当前节点处与操作判断即可

如使用递归下降的方案进行类型检查，递归遍历 AST 进行检查

```
1. class TypeChecker:
2.     def __init__(self):
3.         self.symbol_table = {} # 符号表用于存储变量和函数的类型信息
4.
5.     def check_type(self, ast, target_type):
6.         if ast is None:
7.             return # 空节点
8.
9.         if ast.is_leaf(): # 叶子节点
10.            if ast.type != target_type:
11.                print(f"Type mismatch at line {ast.line}: Expected {target_t
12.                    ype}, got {ast.type}")
13.            return
```

```

14.         if ast.operation == 'assignment':
15.             # 处理赋值
16.             variable_name = ast.children[0].value
17.             assigned_type = self.infer_type(ast.children[1])
18.
19.             # 检查赋值类型匹配
20.             if self.symbol_table.get(variable_name) is not None and self.symbol_table[variable_name] != assigned_type:
21.                 print(f"Type mismatch at line {ast.line}: Variable {variable_name} has conflicting types")
22.             else:
23.                 self.symbol_table[variable_name] = assigned_type
24.
25.         elif ast.operation == 'binary_operation':
26.             # 处理二元操作
27.             left_type = self.infer_type(ast.children[0])
28.             right_type = self.infer_type(ast.children[1])
29.
30.             # 检查操作数类型匹配
31.             if left_type != right_type:
32.                 print(f"Type mismatch at line {ast.line}: Binary operation with mismatched types")
33.
34.             # 递归子节点
35.             for child in ast.children:
36.                 self.check_type(child, target_type)
37.
38.     def infer_type(self, ast):
39.         if ast.is_leaf():
40.             return ast.type
41.         elif ast.operation == 'literal':
42.             return ast.type
43.         elif ast.operation == 'variable':
44.             return self.symbol_table.get(ast.value, None)

```


8.2 指针和引用类型的检查：

编译器检查是否指针类型不兼容等情况。

C++ 编译器在编译过程中进行指针类型兼容性检查，主要涉及到两个方面：隐式类型转换和严格类型匹配。C++ 编译器允许一些隐式类型转换，例如从派生类指针到基类指针，或者从派生类指针到 `void*` 指针。而编译器会检查指针的严格类型匹配，确保在涉及类型转换时不会导致类型不匹配的问题，通常包括对基本数据类型和用户定义的数据类型的匹配检查。

解决方案：在语义分析阶段，编译器会对 AST 进行深入的类型检查。对于指针类型检查和兼容性，编译器会考虑以下几个方面：

1. **隐式类型转换检查：**检查是否存在涉及不同类型的指针之间的隐式类型转换。例如，从一个类的指针类型向上转换为其基类的指针类型。
2. **严格类型匹配检查：**检查各个表达式中的指针是否具有相同的基本类型。例如，检查赋值操作符的左右操作数的指针类型是否一致。
3. **类型转换操作符检查：**对于使用类型转换操作符(如 `static_cast`、`dynamic_cast` 等)的情况，检查转换的目标类型是否合法。
4. **用户自定义类型检查：**对于用户定义的类或结构体，编译器可能需要调用相应的构造函数或转换函数，确保类型兼容性。

8.3 数组越界检查

数组越界检查防止访问时超出数组存储元素个数。在检查时会分为静态分析阶段和动态分析阶段。

在静态分析阶段，编译器建立符号表，其中记录了数组的大小以及其他相关信息。对于静态定义的数组，编译器可以直接获取其大小；对于动态定义的数组，编译器可能需要进行推导或在运行时计算。随后，编译器分析代码中的数组访问操作，检查数组下标的合法性。

而在动态运行时检查阶段，编译器在生成目标代码时插入边界检查代码，以确保每次数组访问时的下标都在数组的有效范围内。如果数组越界，编译器生成相应的异常处理代码或错误提示，以便在程序运行时捕获问题。这一综合的静态和动态检查机制有助于确保程序在执行期间对数组的访问是安全的。

8.4 内存泄漏检查

对于动态分配的内存，进行内存泄漏检查，确保程序正确释放不再需要的内存。

解决方案：内存泄漏检查通常是通过在编译器生成的目标代码中插入检测代码和运行时库来实现的。在 C++ 编译器内部实现内存泄漏检查的过程中，通常采用两个关键策略。

1. 编译器会在生成的目标代码中插入检测代码，用于跟踪动态内存的分配和释放。其中包括记录每次分配的内存块的地址、大小等信息。

2. 编译器依赖运行时库的支持，该库包含实际的内存管理实现和执行检测代码的机制。运行时库在程序启动时初始化，随后负责定期扫描内存管理结构，检查未释放的内存块，并在发现问题时生成相应的警告或记录信息。这一双重策略帮助确保程序在运行时正确释放不再需要的内存，防止潜在的内存泄漏问题。

内存泄露检查开销问题：内存泄漏检查会引入一些运行时开销，可能降低程序性能。因此，在发布版本中，一些编译器提供了关闭内存泄漏检查的选项，以提高执行效率。在开发和测试阶段，开启内存泄漏检查有助于及早发现潜在问题，确保程序的稳定性。

8.5 高级语言的中间代码（IR）

常见的中间代码包括三元式、四元式、SSA 形式、特殊设计的中间代码（如 C++ 的 GIMPLE 等）、字节码和高级中间表示（HIR）等。

其中 SSA 中每个变量在其整个生命周期内只能被赋值一次，可以进行更有效的数据流分析和优化。在 SSA 形式中，每个变量都具有唯一的定义点，这样就避免了在不同的控制流路径中产生多个赋值。一些编译器如 LLVM，使用 SSA 形式的中间代码，例如 LLVM IR。此处介绍 GCC 编译器与对应的 GIMPLE 中间代码形式和 LLVM 编译器。

8.5.1 GCC & GIMPLE

GCC (GNU Compiler Collection) 是由 GNU 项目开发的一套编程语言编译器集合，支持多种编程语言，包括 C、C++、Fortran 等，其支持多平台多操作系统。GIMPLE (GNU Compiler Intermediate Representation) 是 GCC 的一种中间代码表示形式，用于在编译器的优化阶段进行高级语言到机器语言的转换。GIMPLE 是 GCC 内部的一种抽象语法树 AST 的表示形式，它对源代码进行了一定程度的简化和规范化，以便更容易进行优化和代码生成。

GIMPLE 它包括一系列的基本语句，例如赋值语句、条件语句、循环语句等，这些语句对应于源代码中的基本构造块。GIMPLE 还使用上文提到的 SSA 形式，即每个变量只能被赋值一次。这种中间表示形式的设计使得 GCC 能够支持多种编程语言，并在后端进行统一的、高效的优化操作，为编译器提供了更好的可维护性和可扩展性。

8.5.2 LLVM

LLVM 是底层虚拟机的缩写，LLVM 可以在编译、链接、运行时和运行之间的空闲时间为编译器转换提供信息。LLVM 在编译流程中的作用如图所示。

在 LLVM 以前，常用的编译架构一般被分成前端，优化，后端三个部分。在前端，先对源代码进行解析，然后将解析后的源代码转化为抽象句法树；优化部分对与语法树进行等价

转换，从而提高转换效率。后端则需要根据不同的机器生成与之对应的机器码，所以需要为不同语言开发前后端、优化过程等。

LLVM 中间表示代码 IR 是一种静态单赋值语句，IR 是 LLVM 的一种表示形式，其类似于汇编代码的语言，但它又像其他程序设计语言一样具有函数和语句块。IR 代码能反映计算机底层，而且有着良好的可读性。IR 同时还被设计用于过程间优化、运行时优化、程序全局分析等。

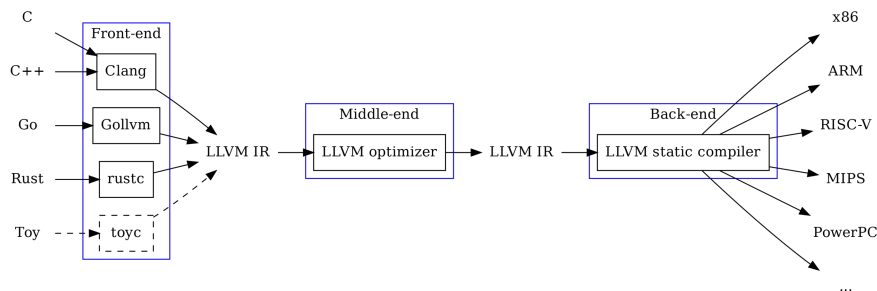


图 27 LLVM 功能示意图

8.5.3 中间代码（IR）生成

经过编译器前端所有处理后，前端最后一步需要生成 IR 代码，在前期的语法分析语义分析模块将根据给定的规则将 MSVL 程序生成四类语法树：分别是自定义类型树、全局变量树、函数树和主语法树。处理顺序依次为自定义类型树、全局变量树、函数树和主语法树。对于主语法树，对其进行深度优先遍历，从最左的叶子节点开始遍历，直到整个转换完成。对简单的语句，例如算术运算、位运算与逻辑运算等，都可以将其直接翻译成 IR 代码。接下来需要进一步对这几种语法树进行处理，

1. **处理自定义类型树：**译器前端可能会处理用户定义的结构体、枚举、联合等自定义类型。每个类型都需要被正确地映射到 IR 代码中，以便后续的引用和使用。
2. **处理全局变量树：**全局变量的初始化和声明需要转换为相应的 IR 代码。这可能涉及到内存分配、初始化值的处理等。
3. **处理函数树：**对于每个函数，编译器需要将其参数、局部变量、函数体等信息转换为 IR 代码。这可能包括栈帧的设置、参数传递、局部变量的内存分配等。
4. **处理主语法树：**在主语法树上进行深度优先遍历，处理各种语句和表达式。对于简单的算术、位运算和逻辑运算，编译器可以直接生成对应的 IR 代码。对于更复杂的控制结构，如条件语句和循环，编译器需要将其转换为对应的 IR 表示，可能涉及到基本块的生成和控制流的调整。

九 总结与收获

9.1 成果概述

本项目中，我们使用 python 语言完成语义分析和中间代码生成功能，它是整个编译器系统的核心部分之一。在课程的理论学习基础之上，我们积极进行动手实践，实现了一个具有交互功能的中间代码生成器：

1.词法分析器模块 我们将词法分析并生成单词流、求给定文法中终结符、非终结符。

2.语法分析器模块 我们调用词法分析器的分析结果，计算产生式的 First 集、计算 LR(1) 项目簇、构建对应 ACTION 表和 GOTO 表等诸多课程理论知识进行代码实现，完成了类 C 语言的词法分析和语法分析过程。

3.语义分析与中间代码生成模块 我们调用语义分析模块的相关函数，在分析语法的基础上继续对输入代码进行类型检查、语义错误检测等处理，并且以四元式的形式生成中间代码，完成了图 1 中编译器的前三步。

4.模块整合连接 我们小组将以上两大模块的代码进行了整合和连接，确保它们能够协同工作，也方便前端封装。我们设计了清晰的接口和数据结构，使得模块之间的通信和数据传递变得简单而高效。通过仔细的代码审查和测试，我们确保了整个系统的稳定性和正确性。

5.前端展示模块 为了提供一个良好的用户体验，我们小组也设计了前端展示部分。我们充分了用户的操作逻辑，通过 disable 按钮显示的方式引导用户正确操作，保证所访问的过程数据文件的正确性。并且编译器提供了一些额外的功能，如代码高亮显示、错误提示等，以帮助用户更好地理解和分析代码。

9.2 项目收获

本次实验在第一次实验的语法分析基础上实现了语义分析静态语义检查、中间代码生成。并最终实现了将四元式和语义分析过程展示功能和第一次实验 UI 接口完成对接，两次实验共同设计实现了编译器的前端构建工作。

从项目结构构思角度 开始前我们进行了充分的讨论和研究，以确定整个项目的结构和框架。我们首先重新复盘了语法分析器实验的各函数和模块功能，其次细致了解了中间代码生成的基本概念和原理，然后根据实验要求和我们的理解，共同设计了项目的整体结构，在明确了各个模块的功能和相互之间的依赖关系之后，项目推进更有目的性。

静态语义检查实现 本次实验添加实现了语句的翻译功能，从语义推导式出发设计了较为规范的分支跳转结构。在不同类型语句的处理过程中注意维护了分析栈和语义栈的转换过程，并根据语句特征（部分语句需要回填和考虑真链假链）产生对应四元式。

前端展示与用户逻辑 本次实验在上一次实验的基础上添加了 FirstSet 展示、四元式展示以及语义分析栈展示的功能。同时为部分页面添加了文本框提示信息，能够为用户交互提供必要的引导。同时本编译器前端展示部分充分考虑到编译过程流程和用户交互逻辑，用户会在引导下按照最简编译流程进行交互，保证编译过程产生的文件内容不会混淆。

通过这个实验，我们在实践中加深了对语义分析和中间代码生成原理的理解，也在团队合作中提高了沟通和协作能力，同时为我们下学期继续开发编写整个简易的编译器的工作打下了坚实的基础。

参考文献

[1]Chen,H., in,L.chun, & ping, T. qing.(2000).程序设计与编译原理.国防工业出版社.

[2]“Online C Compiler,” TutorialsPoint. https://www.tutorialspoint.com/compile_c_online.php

- [3]M. Godbolt, “Compiler Explorer.” <https://godbolt.org/>
- [4]“Online C Compiler.” <https://www.programiz.com/c-programming/online-compiler/>
- [5]“Compiler construction tools,” GeeksforGeeks, Apr. 13, 2023.
<https://www.geeksforgeeks.org/compiler-construction-tools/>
- [6]“Principles and Practice of Compiling | Department of Computer Science.”
<https://www.cs.iastate.edu/courses/com-s-540>
- [7]“Bison 3.8.1.” <https://www.gnu.org/software/bison/manual/bison.html>
- [8]“解决移进/规约冲突_移进规约冲突例子-CSDN 博客.”
https://blog.csdn.net/iamagoodguy_/article/details/48499591